▢ Lecture 1

Welcome to Foundations of Computing
This course transforms you from a passive technology user into an active developer by teaching you how apps and digital services actually work behind the scenes. Rather than just using technology like Google Pay, YouTube, or Google Maps, you will learn to write the instructions that power these applications.

Course Overview
The course is structured around four core learning objectives:

Computing Fundamentals: Understand what computing is, how it thinks, and what components it is made of
Programming with Python: Write real, functional code using Python, the world's most popular programming language

Data Visualization: Transform raw numbers and data into meaningful charts and visual stories
Real-World Project – Sentiment Analysis: Build a machine learning application that analyzes text (such as customer reviews) and determines whether the sentiment is positive or negative
Why Python?
Python is chosen as the primary language because it:

Is the most popular programming language globally
Powers AI, data science, and automation projects
Reads almost like English, making it accessible to beginners
Does not require advanced mathematical knowledge—only curiosity and willingness to learnLearning Outcomes
By the end of this course, you will:

Understand the fundamentals of how computing actually works
Build a real sentiment analysis project that businesses use to evaluate customer feedback
Develop skills that are in high demand by companies across industries
Gain practical experience that opens career opportunities in technology and data science

Lecture 2
What is Computing?
Computing is fundamentally different from computers as devices. Computing is the abstract process of solving problems through a well-defined series of steps, taking information as input, processing it, and producing useful results. This concept predates modern electronic computers by centuries.

Historical Examples of Computing:

Ancient Abacus (3000 BC): Merchants used this tool to perform addition and subtraction, demonstrating that computing existed before electronic machines
Navigation Tables (1600s-1800s): Sailors used pre-calculated tables to determine their positions at sea by processing coordinates through mathematical tables
Human Computers (World War II): Women performed complex arithmetic calculations to determine artillery trajectories, and their job title was literally "computers"—the term referred to humans before it referred to machines
These examples illustrate that computing is a methodology, not dependent on any particular technology.

The IPO Model: Foundation of All Computing

The IPO Model (Input-Process-Output) is the fundamental principle underlying every computing process:

Input: Raw data or information provided to the system
Process: The execution of algorithms and logical operations on the input
Output: The meaningful result produced after processing
This model applies universally—from simple calculators to artificial intelligence systems. Understanding this model is essential because it appears in every digital application and deviceReal-World Examples of the IPO Model
Smartphone Applications:

Input: User touches and swipes on the screen (e.g., selecting food items in a delivery app)
Process: App logic executes based on user

actions (e.g., adding items to cart, processing payments)
Output: Visual feedback appears on the screen for each action
Google Search Engine:

Input: Natural language query typed into the search bar
Process: Algorithm searches billions of web pages in milliseconds, ranking them based on keyword relevance, page speed, mobile friendliness, and domain trustworthiness
Output: Ranked search results displayed on the screen
UPI (Unified Payments Interface):

Input: Amount to transfer and UPI PIN
Process: Bank verifies identity, checks account balance, debits amount from sender's account, credits amount to receiver's account

Output: Confirmation notifications sent to both parties

Google Maps:

Input: Current location and destination address
Process: Shortest path algorithm evaluates hundreds of possible routes, considering current traffic, roadblocks, and other factors
Output: Recommended route displayed with estimated travel time

YouTube Recommendation Engine:

Input: Video watched by user, watch duration, and sections skipped
Process: Algorithm compares user behavior with millions of other users globally, identifying patterns in viewing habits (a machine learning process)
Output: Personalized video recommendations related to user's interests

Smartphone Autocorrect:

Input: Letters and words typed by the user
Process: Algorithm analyzes typed letters and previous words, predicts next word based on patterns learned from millions of user sentences
Output: Suggested next word for user selectionComputing vs. Computer: A Critical Distinction
Computing is the abstract intellectual concept of problem-solving through defined, step-by-step procedures. Computer is the physical electronic tool that executes computing processes with speed and precision.

Key differences:

Computing existed for thousands of years before electronic computers were invented
Computing can be performed with pen and

paper, an abacus, or any computational tool
Computers are fast executors of computing processes, but the intelligence and problem-solving methodology comes from humans who design the algorithms
When asked to define computing, avoid mentioning computers—focus on the process and methodology
The Four Pillars of Computing
All computing processes depend equally on four essential pillars. The failure of any single pillar causes the entire computing process to fail:

Data: Raw, unprocessed facts and observations that serve as the foundation for computing
Algorithm: Well-defined set of instructions that process data
Hardware: Physical electronic components that execute instructions
Software: Programs and code that implement

algorithmsUnderstanding Data: The Foundation of Computing
Data is any raw, unprocessed fact or observation that can be recorded and stored. By itself, isolated data has no inherent meaning—it requires context to become valuable.

Data vs. Information:

A number like "38.5" is raw data with no meaning
"Patient's temperature is 38.5 degrees Celsius" provides context and becomes information
Raw data requires context to be meaningful and useful
Examples of Data with and without Context:

"42" = meaningless raw data
"Age = 42" = meaningful information providing context
"Mumbai" = just a word

"Place of birth = Mumbai" = meaningful information with context
Data in Modern AI Systems:

Data is the lifeblood of artificial intelligence. Advanced models require significantly larger datasets than traditional machine learning models. Key principles:

More advanced models require more training data
Data enables systems to learn, adapt, and make predictions
Machine learning models use training data to identify patterns
Better quality and quantity of training data produces better prediction results
Without data, even the most sophisticated algorithms are completely uselessThe Data-Information-Knowledge-Wisdom Chain

Data progresses through a hierarchy as it gains meaning and application:

Data → Information → Knowledge → Wisdom

Clinical Example: Patient Temperature Analysis

Data: 38.5 (raw number)
Information: Patient's temperature is 38.5°C over four days showing an upward trend, peaking on day four (context and pattern recognition)
Knowledge/Insights: This fever pattern is consistent with viral infection, likely to resolve in 3-5 days (analysis and inference based on patterns from similar cases)
Wisdom: Doctor prescribes specific treatment based on pattern recognition from thousands of similar cases (decision-making and action)
This progression demonstrates how raw data transforms through processing, analysis, and

decision-making into wisdom that drives actions.

Key Takeaways

Computing is about solving problems through well-defined steps, not about machines

The IPO model (Input-Process-Output) is universal across all computing applications

Every application on smartphones, search engines, maps, and AI systems operates on this fundamental principle

Computing requires four essential pillars: data, algorithm, hardware, and software

Data must be contextualized to become meaningful information

Understanding computing principles is essential before learning to program

Lecture 3

Understanding Data: Classification and Organization

Overview

Data is classified into various types to determine appropriate processing methods and analysis techniques. Understanding these classifications is fundamental to computing and data analysis, as each type requires different handling approaches and operations.

Main Classifications of Data

1. Structured vs. Unstructured vs. Semi-Structured Data

Structured Data

Organized into rows (records) and columns (fields) in a tabular format

Each field has a defined data type (number, text, date, yes/no)

Enables easy searching, sorting, filtering, and analysis operations

Examples: CSV files, Excel spreadsheets, Google Sheets, database tables, bank transaction records, student marksheets, hospital patient registers, e-commerce order tables
Represents a smaller portion of global data
Unstructured Data

Does not follow a predefined format or organization
Cannot be neatly placed into rows and columns
Comprises approximately 80% of all data generated globally
Types include:
Text: emails, messaging chats, news articles, customer reviews, social media posts
Images: photos, medical x-rays, satellite imagery, ID card scans
Audio: voice calls, podcasts, music files, voice notes
Video: CCTV footage, YouTube videos, recorded

lectures
Sensor data: raw IoT device readings without labels
Processing requires specialized tools and techniques like transformers for natural language processing and computer vision for imagesSemi-Structured Data

Falls between structured and unstructured data
Has some organizational markers but not in tabular format
Examples: JSON files (used in web APIs), XML files, HTML pages, emails (with fixed headers but free-form body)
Allows for organization and analysis despite lacking rigid row-and-column structure
2. Data Across Time: Cross-Sectional, Panel, and Time Series Data
Cross-Sectional Data

Collected from multiple subjects (people, cities, companies) at a single point in time
Captures a snapshot at one moment
Allows comparison across subjects but cannot track individual changes over time
Examples: election polls, consumer surveys, Amazon product feedback, census data, medical studies comparing patient groups
Use case: survey of 500 students in Delhi on June 1, 2025
Panel Data (Longitudinal Data)

Tracks multiple subjects over multiple time periods in sequence
Combines breadth of cross-sectional data with multiple time points
Enables both cross-subject comparison and over-time trend analysis
More information-rich than cross-sectional or time series data alone

Examples: monthly marks of students tracked over six months, NREGA employment records (collected yearly), IMF macroeconomic data across countries over years, clinical drug trials, school performance tracking
Allows analysis of which subject improved most over time
Time Series Data

Collected for a single subject at multiple points in time in sequence
Focus is on tracking one entity over different time instances
Time is the key dimension, and sequence matters
Reveals trends, cycles, and sudden changes
Examples: daily closing stock prices, weather forecasting data, ECG heartbeat monitoring, website traffic, epidemic spread and case counts, smartphone step counter data,

electricity meter readings, financial market data
Note: Distinguished from panel data by tracking only one subject rather than multiple subjects3.
Quantitative vs. Qualitative Data
Quantitative Data (Quant Data)

Numerical and measurable
Can be counted or measured
Two subtypes:
Discrete: Countable whole numbers (e.g., number of students: 30, number of goals: 3, number of apps: 47)
Continuous: Any value within a range including decimals (e.g., height: 172.4 cm, temperature: 36.8°C, time: 9.5 seconds)
Mathematical operations possible: addition, subtraction, mean, median, standard deviation, correlation
Advantages: quick and easy to analyze, less prone to errors and biases, enables identification

of trends and patterns, results can be generalized with representative sampling
Collection method: surveys
Answers questions like: what, how often, how many, how much
Qualitative Data (Qual Data)

Categorical and non-numerical
Represents labels, categories, qualities, and opinions
Two subtypes:
Nominal: Categories with no natural order (e.g., blood group A/B/AB/O, city of birth, phone brand). Operations possible: mode, frequency count, chi-square test
Ordinal: Categories with meaningful order but unequal gaps (e.g., Amazon star ratings 1-5, education level, customer satisfaction: poor/fair/good/excellent)
Includes natural language text, feedback, and

opinions
Advantages: provides in-depth insights and contextual information
Collection method: interviews
Answers questions like: how, why
Limitations: analysis more prone to researcher bias, requires thematic or content analysis, interpretation is subjective4. Primary vs. Secondary Data
Primary Data: Collected directly from the source for a specific purpose
Secondary Data: Already collected data repurposed for a different analysis
Importance of Data Classification
Understanding data types is crucial for programmers and data analysts because:

Type of data determines which operations are meaningful and valid
Cannot perform mean or median on nominal or

ordinal data
Averaging city of birth would yield no meaningful insight
Proper classification ensures appropriate analysis methods and reliable conclusions

Lecture 4

Introduction to Computing and Programming Foundations

Dr. Raman Pathak, PhD in Information Systems Management from IIT Kanpur, introduces the course on foundations of computing with a comprehensive overview of his expertise spanning both technology and management domains, including experience with AI adoption in the BFSI sector (Banking, Finance, Securities, and Insurance).

Course Overview and Evaluation Scheme

Class Participation: 10% weightage through assignments submitted after lectures
Quizzes: Best 2 out of 3 quizzes recorded, 15% each (30% total)
Final Exam: 60% weightage in offline mode
Programming Language: Python
Development Platform: Google Colab (cloud-based IDE requiring no local installation)
Why Learn Programming in the Age of AI and LLMs?
The Vibe Coding Phenomenon
Vibe coding—using AI language models like ChatGPT, Gemini, or Claude to generate code through natural language prompts—is gaining popularity. However, relying solely on vibe coding poses significant challenges:

Debugging Complexity: While code generation is easy, identifying and fixing errors becomes extremely difficult without programming

knowledge
Token Costs: Each interaction with LLMs consumes expensive tokens; users unfamiliar with programming repeatedly request AI assistance, exhausting budgets quickly
Code Quality Issues:
Spaghetti code (disconnected, unstructured code)
Security vulnerabilities
Non-existent library imports
Logical errors in complex applications
Production Failures: When AI-generated scripts fail in live production environments, developers without programming knowledge cannot troubleshoot issues
Business and Regulatory Accountability
Developers and managers remain legally and financially accountable for code failures, regardless of whether code was AI-generated
Regulatory bodies (RBI for banking, SEBI for

financial markets, IRDAI for insurance) impose heavy penalties for code violations

Real-world example: PTM faced bans for data security mismanagement in applications

Current Reality of LLM Capabilities

LLMs excel at generating simple code and 80% of prototype work

LLMs struggle with complex programming, architecture design, and integration with legacy systems

Recent industry trends show major tech companies (Google, Meta, Open AI) actively rehiring engineers after initial AI-driven layoffs, recognizing that tokens are expensive and human expertise remains irreplaceable

Programming knowledge is essential for the next 10+ years; AI is a powerful assistant, not a replacement

What is Computing?

Definition and Core Concept

Computing is the process of using well-defined logical steps to solve problems by collecting information, processing it, and producing useful results. It is not dependent on computer technology but rather on the delegation of repetitive, logical tasks to machines.

Historical Context
During World War II, "computers" were women working at desks manually calculating artillery trajectories using pencil and paper
The core concept remains unchanged: delegating repeatable logical steps to a mechanism to solve problems at speeds humans cannot achieve
IPO Model (Input-Process-Output)
The fundamental framework for all computing tasks:

Traditional IPO Model (30 years ago):

User provides input via keyboard
CPU processes data
Output displayed on screen
Enterprise IPO Model (Current Era):

Input: High-velocity, high-variety, high-volume data streams

Real-time sensor data
Structured and unstructured data
Multiple data formats (text, audio, video, sensor readings)
Data measured in zettabytes (10^{21} bytes)
Process: Automated algorithms powered by cloud infrastructure executing logical steps billions of times per second, reducing computational costs to near zero

Output: Actionable insights and business

decisions translated into revenue growth or cost reduction

Real-World Computing Examples

Weather Forecasting

Input: Billions of live data points (wind speed, precipitation, humidity) from global satellite networks and ground stations every hour

Process: Physics-based and AI computer models analyzing data streams

Output: Local weather forecasts for coming days

Business Impact: Critical for agriculture, logistics, and disaster management

Search Engines (Google/Gemini)

Input: User search query

Process: Algorithm mapping query relevance across billions of indexed web pages

Output: Most relevant web results ranked by relevance

Business Impact: Advertising revenue and user

engagement

Unified Payments Interface (UPI)

Input: User PIN and transaction amount

Process: Request routed through payer bank → NPCI server → payee bank with multiple authentication checkpoints

Output: Transaction completion notification

Timeline: Completed within 3-4 seconds

Business Impact: Enables digital payment ecosystem at scale

Google Maps

Input: Satellite data, GPS signals, real-time traffic conditions, current road status

Process: Route optimization algorithms considering multiple variables

Output: Optimal routes displayed on device screens

Business Impact: Navigation services, location-based advertising

Recommendation Engines (Amazon, YouTube)

Input: User browsing history, purchase data, viewing patterns
Process: Clustering users into segments; probability-based prediction models
Output: Personalized product or video recommendations
Business Impact: Increased user engagement and sales conversion
Uber Platform
Input Components:

Driver GPS locations (sensor data)
User ride requests
Local traffic API data
Weather information
Real-time demand metrics
Processing Algorithms:

Dynamic pricing algorithm: Adjusts fares based on supply-demand balance

Route optimization: Calculates most efficient paths
Matching engine: Connects drivers with riders within 500-meter radius
Output:

Driver location and arrival time (ETA)
Dynamic pricing with surge charges during high demand
Transaction execution
Business Impact: Maximizes platform yield and operational efficiency; surge pricing (e.g., during rain) balances supply and demand

The Role of Cloud Computing
Traditional computing required expensive on-premise server infrastructure. Cloud providers (AWS, Google Cloud Platform, Microsoft Azure) deliver computing power over the internet, enabling:

Processing of massive data streams globally
Scalability to billions of simultaneous requests
Dramatic reduction in infrastructure costs
Real-time data processing and response delivery

Key Takeaways

Programming knowledge remains essential despite AI advancement; it enables code auditing, debugging, and professional accountability

Computing is fundamentally about solving problems through defined logical steps, whether performed by humans or machines

The IPO model applies across all business domains and technology platforms

Emerging technologies amplify the need for human expertise in architecture, design, and system oversight

AI should be viewed as a powerful assistant to enhance programmer productivity, not as a

replacement for human judgment and technical understanding

Lecture 5

Foundations of Computing: Data Classification and Introduction to Algorithms

Overview of the Lecture

This lecture covers the fourth classification of data (primary vs. secondary data and metadata) and introduces the second pillar of computing: algorithms. The session builds on previous discussions of data classifications based on format, time, and nature.

Data Classification: Origin-Based (Fourth Classification)

Primary Data

Primary data is data collected firsthand by researchers for a specific purpose and is original, not previously published.

Collection Methods:

Creating and distributing surveys (e.g., Google Forms via email, WhatsApp, social media)
Conducting experiments and recording observations
Collecting sensor readings from deployed IoT (Internet of Things) devices
Arduino microcontrollers and weather stations collecting physical data
Advantages:

Tailored to exact research questions and purposes
Reliable source with known collection methods
Direct control over data gathering process
Disadvantages:

Time-consuming (requires survey creation, distribution, and response collection)

Expensive, requiring financial and human resources
Often limited to smaller sample sizes due to cost and time constraintsSecondary Data
Secondary data is data collected by someone else for a different purpose, which is then reused for new analysis.

Sources:

Bloomberg and Prolific databases
SEBI (Securities and Exchange Board of India) data
IRDAI (Insurance Regulatory and Development Authority) data
NSSO (National Sample Survey Office) survey data
WHO Global Health Statistics
Government census data
Wikipedia structured data

Advantages:

Large-scale data collection possible due to organizational resources
Often available for free or at minimal cost
Authoritative and controlled by established organizations
No need to conduct individual data collection
Disadvantages:

May not perfectly match specific research questions
Data quality depends on the original collector
Limited control over how data was collected
Possible subscription or licensing fees required for accessMetadata
Metadata is data about data—it describes the characteristics and context of data without being the actual data itself.

Examples:

Photo metadata: File name, file size, creation date, GPS location, camera model, image resolution
Database metadata: Column names, data types, primary keys, foreign keys, number of rows and columns
Library catalog information: Book title, author name, publication details, availability status, subject area
WhatsApp messages: Sender ID, recipient ID, timestamp, read status, device type
Importance:

Critical for organizing, finding, and governing large datasets
Reveals information without exposing actual content
Essential for data management and system

organization

Introduction to Algorithms

Definition and Core Concept

An algorithm is the intellectual core of computing —a finite, unambiguous, step-by-step procedure for solving a well-defined problem. Algorithms exist independently of computers; computers are simply machines that execute algorithms very rapidly.

Historical Context

The term "algorithm" derives from Muhammad Ibn Musa Al-Qarizmi, a 9th-century Persian mathematician

His work on arithmetic and algebra introduced systematic problem-solving methods

The word "algebra" also originates from his book title

Algorithms existed before modern computers (e.g., women "computers" calculating artillery

trajectories during World War IIImportance of Algorithm Quality

The quality of computing depends on algorithm quality, not just hardware power

A well-optimized algorithm can work effectively even with less computing power

A bad algorithm produces wrong or slow results regardless of hardware capability

Researchers in the 1990s conducted AI research with limited computing power by optimizing algorithms

Simple Algorithm Example: Determining Odd or Even Numbers

Start

Read number n

Divide n by 2 and check remainder

If remainder = 0: print "even"

If remainder ≠ 0: print "odd"

End

Five Essential Properties of Algorithms

1. Finiteness

   Algorithm must terminate after a finite number of steps

   Cannot run forever or loop infinitely

   Example of bad algorithm: "Divide n by 2 until n equals 0" (will never reach exactly 0)

2. Definiteness (Unambiguity)

   Every instruction must be precisely and clearly defined with no room for interpretation or subjectivity

   Same step must produce identical results every time

   Bad example: "Add a little bit of salt" (subjective)

   Good example: "Add exactly 5 grams of salt" (precise and objective)

3. Input

   Algorithm may have zero or more inputs

Inputs are values given before or during execution

Examples:

Finding maximum of two numbers requires 2 inputs

Computing factorial requires 1 input (the number)

Sorting a list requires the list as input

Displaying "Hello World" requires 0 inputs4. Output

Algorithm must produce at least one output (result of computation)

Without output, the algorithm is pointless

Examples:

Maximum algorithm outputs the larger number

Factorial algorithm outputs the factorial value

Sorting algorithm outputs the sorted list

Navigation algorithm outputs step-by-step route

4. Effectiveness

   Every step must be basic enough to be performed by a person using pen and paper (in principle)

   No step should require infinite resources or be physically impossible

   Resource requirements should be optimal and practical

   Bad examples:

   Finding the largest prime number (numbers are infinite)

   Asking 1 billion users simultaneously (requires infinite resources)

   Good examples:

   Comparing two numbers to find which is bigger (basic, executable operation)

   Common Algorithmic Structures

   Do-While Loop: Executes code then checks condition

If-Else Statement: Executes different code based on condition
While Loop: Checks condition before executing code
All algorithmic structures follow the same foundational principles and will be covered in detail during Python programming lessons.

Lecture 6

Hardware: The Physical Foundation of Computing

Hardware represents the physical foundation that enables all computing operations. Understanding hardware is essential for writing better software and answering critical questions about why memory management matters, why certain loops are slow, and how data persists after power loss.

Overview of the Four Pillars of Computing

The lecture focuses on Pillar 3: Hardware, following previous discussions on data (Pillar 1) and algorithms (Pillar 2). The fourth pillar, software, is also introduced briefly.

The Role of Hardware in Artificial Intelligence

Limited Progress in Early AI Research: AI research began in the 1990s but progressed slowly due to insufficient computing power

GPU as the Game Changer: The explosion of AI over the last decade resulted from Graphics Processing Units (GPUs), not just algorithmic breakthroughs

Parallel Processing Power: GPUs are designed for mass parallel processing, breaking complex tasks into thousands of smaller tasks solved simultaneously

The 2012 Breakthrough: Scientists developed AlexNet for computer vision using GPUs,

demonstrating the technology's potential to the world

Modern AI Applications: GPUs are now essential for training neural networks in:

Large language models (ChatGPT)

Image generators (DALL-E, Midjourney, Stable Diffusion)

Video AI applications

NVIDIA's Leadership: NVIDIA leads GPU development through its CUDA (Compute Unified Device Architecture) platform, with Python libraries like PyTorch (META) and TensorFlow (Google) enabling GPU-accelerated code

Input Devices: Translating the Real World into Digital Signals

Input devices convert actions and information from the real world into digital signals that computers can process. Every interaction with a computing device begins with an input device.

Common Input Devices
Keyboard and mouse
Touchpad and touchscreen
Microphone and camera
Sensors (convert physical signals into electrical signals)
Sensors as Input Devices
Sensors in everyday devices detect physical phenomena and convert them into data:

Sensor Function Real-World Applications
Accelerometer Detects movement and orientation Screen rotation (portrait to landscape), step counting (Google Fit, Strava)
GPS Receives satellite signals for precise location Navigation apps (Google Maps, Uber, Ola, Swiggy, Zomato)
Gyroscope Detects rotational movement Gaming (car racing games), navigation, camera

stabilization
Fingerprint Sensor Captures biometric data Phone authentication and device locking
Barometer Measures air pressure Altitude and weather prediction apps
Temperature Sensor Monitors device temperature Overheating warnings during intensive computing
Light Sensor Detects ambient light Automatic screen brightness adjustment based on time of day
Proximity Sensor Detects proximity to ear Automatic screen turn-off during phone calls
Processing Units: The Brain of Computing
CPU (Central Processing Unit)
The general-purpose brain of the computer that executes instructions sequentially at high speed.

Key Components:

ALU (Arithmetic Logical Unit): Performs arithmetic operations (addition, subtraction, multiplication, division) and logical operations (AND, OR, NOT, comparisons)
Control Unit (CU): Orchestrates instruction fetching from memory, decoding, and directing operations to ALU and other units; decides which instruction executes next when multiple applications run simultaneously
Performance Metrics:

Modern CPUs execute 3 to 5 billion instructions per second
Speed measured in GHz (gigahertz)
Multiple cores enable simultaneous task processing (4, 8, 16, or up to 24 cores in modern processors)
Role in Programming: When running a Python program, the interpreter translates high-level code into machine instructions that the CPU

executes.

GPU (Graphics Processing Unit)
Designed differently from CPUs, optimized for parallel processing of large datasets.

Architecture Differences:

CPU: Few powerful cores (4-24), optimized for complex sequential tasks
GPU: Thousands of small, simple cores, optimized for performing the same operation on millions of data points simultaneously
Original Purpose: Graphics rendering and video game pixel color and position calculations

Current Applications in AI:

Training neural networks for computer vision and machine vision

Large language models and image generators
Performing the same mathematical operations on billions of numbers simultaneously
Memory: Where Data Resides
Memory stores data currently being used by the computer. Two primary types exist with different characteristics.

RAM (Random Access Memory) - Fast and Temporary
Characteristics:

Volatile: Contents are lost when power is cut
Fast: Data accessed in nanoseconds, much faster than storage
Limited Storage: Typical phones have 6-12 GB; laptops have 8-32 GB
Function: The computer's working memory, similar to a physical desk while studying

Usage: When apps are opened, they load from storage into RAM for execution. Variables, lists, and objects exist in RAM during runtime and are cleared when programs close.

ROM (Read-Only Memory) - Permanent and Immutable
Characteristics:

Non-volatile: Instructions remain intact even without power
Permanent: Instructions burned at factory and cannot be changed
Primary Memory: Like RAM, part of the main memory system
Contents:

Firmware and bootloader (program running before OS starts)
Fixed instructions that ensure system

initialization
Secondary Storage - Permanent Data Storage
Where data lives permanently, even when devices are powered off. This is where documents, images, videos, Python files, music, and databases are kept.

Types of Secondary Storage:

SSD (Solid State Drive): Modern replacement for hard disk drives; faster access speeds
HDD (Hard Disk Drive): Older technology, now less common
Cloud Storage: Internet-based storage on remote servers, accessible via internet
USB Drives and SD Cards: Portable storage for data transfer
Optical Disks: Older storage media
Output Devices: Translating Results to Humans
Output devices transfer computer results back

to the physical world in forms humans can understand and use.

Common Output Devices:

Display/Monitor: Screen output
Printer: Physical document output
Speakers: Audio output
Haptic Feedback: Vibration response (e.g., smartphone keyboard vibration)
Actuators/Motors: Mechanical output devices
Function: Convert digital data processed by the CPU into forms humans can perceive and use.

System Software: The Operating System
System software manages hardware resources and provides a platform for other software to run. The operating system is the master program that starts first, runs always, and never stops until shutdown.

Common Operating Systems

Operating System Primary Use Key Features

Windows 11 Personal computers Most widely used desktop OS; dominates enterprise

macOS Sequoia Apple computers Tight hardware-software integration; popular with developers

Linux Servers, IoT Open-source; runs 96% of world's web servers; high adoption

Android Smartphones 72% global smartphone market share; open-source base

iOS Apple phones/tablets Proprietary; closed ecosystem; known for security and performance

Chrome OS Chromebooks Cloud-first; runs Android apps; everything stored and accessed via internet

Core Functions of Operating Systems

Process Management: Decides which programs run and for how long on the CPU; manages

dozens of simultaneous processes with millisecond-level switching

Memory Management:

Allocates RAM to programs
Ensures programs don't interfere with each other
Reclaims memory when programs close
File System Management:

Organizes storage into files and folders
Provides consistent methods for programs to save and load data
Independent of underlying hardware
Device Drivers:

Translates generic instructions into specific hardware commands (printer, Wi-Fi, touchscreen)
Enables phones to work with thousands of different peripherals

Required for peripherals to function properly with devices
Security Management:

Controls which programs and users access which resources
Prevents apps from accessing other apps' private data
Defines permissions for different applications
User Interface:

Graphical User Interface (GUI): Desktop icons, windows, touchscreen interaction (Windows, macOS)
Command Line Interface (CLI): Terminal/shell scripting for file and system management
Role as Intermediary
When Python programs read files or print output, they request the operating system to perform these actions. The OS acts as the intermediary

between code and hardware.

Utility Software: System Maintenance and Protection

Utility software maintains, optimizes, and protects the computing system itself. Users interact with utilities when system management is needed.

File Management Utilities

Tools for creating, copying, moving, renaming, and deleting files and folders:

GUI Tools: Windows Explorer, macOS Finder

Command Line Tools: ls, cp, mv, rm (Linux/macOS), dir, copy, move (Windows)

Disk Management Utilities

Disk Cleanup: Removes temporary files, browser caches, and unused installers

Defragmentation: Reorganizes fragmented files on SSDs to improve access speed

Disk Partitioning: Divides storage into logical sections (e.g., C drive for OS, separate drives for data)

Disk Checking: Scans for bad sectors and file system errors; identifies corrupt storage devices

Security Utility Software
Antivirus and Anti-malware: Scan for and remove malicious software using signature databases and heuristics (examples: Kaspersky, Norton, McAfee)

Firewall: Monitors incoming and outgoing network traffic; blocks unauthorized connections

Encryption Tools: Protect data through full-drive

encryption

BitLocker (Windows)
FileVault (macOS)
Renders data unreadable without correct key
VPN Clients: Encrypt network traffic; mask IP addresses; redirect internet requests from different locations for privacy

Compression and Archiving Utilities
Lossless Compression: Reduces file size with identical data restoration capability

ZIP files, 7Z files, TAR files
Entire folders compressed into single files
Lossy Compression: Permanently discards some data for smaller file sizes; ideal for media files

JPEG (images): Removes invisible pixel

variations
MP3 (audio): Removes inaudible frequencies
Significantly smaller file sizes than uncompressed originals
Application Software: User-Focused Programs
Application software is built to help users accomplish specific tasks. Every app on phones, every program on laptops, and every website is application software.

Productivity Applications
Word Processors: Create, edit, and format text documents

Microsoft Word (Windows)
Google Docs
Spreadsheet Software: Organize, calculate, and visualize tabular data

Microsoft Excel (Windows)

Google Sheets
Support formulas as algorithms
Enable data sorting, pivot tables, and computational analysis
Presentation Tools: Structure and display visual content

Microsoft PowerPoint
Google Slides
Database Tools: Create, query, and manage structured data

MySQL Workbench
Excel tables
Store data in rows and columns for easy querying
Key Concepts Review
Hardware Components Hierarchy:

Input Devices (sensors, keyboard, mouse) → 2.

Processing Units (CPU, GPU) → 3. Memory (RAM, ROM, Storage) → 4. Output Devices (display, printer, speakers)
Memory Hierarchy (Speed vs. Permanence):

Fastest and most temporary: CPU registers
Very fast, temporary: RAM
Very fast, permanent: ROM
Slower but permanent: Secondary storage (SSD, HDD)
Software Layers:

System Software (OS) - manages hardware
Utility Software - maintains and protects system
Application Software - helps users accomplish tasks
Show less
Discussions
Share program-related discussions with your peers.

Lecture 7

Overview of Application Software and Programming Fundamentals

This lecture covers the various types of application software, the distinction between interpreters and compilers, and an introduction to Python programming as the foundational language for the course.

Types of Application Software

Communication Software

Email Clients: Software like Gmail and Outlook that enable asynchronous communication using SMTP (Simple Mail Transfer Protocol) and IMAP (Internet Message Access Protocol)

Instant Messaging: Applications such as WhatsApp, Telegram, and Slack that allow real-time messaging with end-to-end encryption; Slack is used in corporate environments for employee collaboration

Video Conferencing: Platforms like Zoom and Google Meet that facilitate real-time audio and video communication using compression and synchronization algorithms

Web Browsers

Parse and render HTML (Hypertext Markup Language), CSS (Cascading Style Sheet), and JavaScript

Manage multiple simultaneous network connections

Provide developer tools for programmers

Examples include Google Chrome, Mozilla Firefox, and Safari

Entertainment Software

Media Players: Applications like VLC that decode and play compressed audio and video formats in real time

Audio Streaming: Services like Spotify that stream compressed audio files

Gaming: Computationally intensive applications

featuring physics simulation, 3D rendering, AI for non-player characters, and network synchronization
Streaming Platforms: Services such as YouTube, Netflix, and Prime Video that deliver video content on demand
Specialized and Professional Software
Professional software serves specific objectives across various domains:

Data Science: Jupyter Notebook for Python programming; uses libraries like Pandas, NumPy, and scikit-learn with algorithms based on statistics and machine learning models
Design and CAD: Tools like AutoCAD, Figma, and Photoshop; employ vector mathematics and rendering algorithms with libraries such as PIL (Pillow) and Cairo
Healthcare: Hospital information systems and PACS (Picture Archiving and Communication

Systems); utilize libraries like scikit-learn and OpenCV
Finance: Bloomberg Terminal and trading platforms; use time series analysis and risk models with libraries like Pandas and statsmodels
Engineering: MATLAB and SolidWorks for physical design and 3D printing; employ numerical methods and simulation using NumPy and SciPy
Programming Language Software
Integrated Development Environments (IDEs)
An IDE is an all-in-one tool that combines:

Code editor for writing and editing code
Code runner for executing programs
Debugger to identify errors and their locations
Version control capabilities
Examples: PyCharm, VS Code, and Google Colab (cloud-based)

Source Code Editors
Simple tools like Notepad, Sublime Text, and VS Code that allow code writing and editing but require separate programs for execution and debugging.

Interpreter vs. Compiler
Aspect Interpreter Compiler
Languages Python, JavaScript C, C++, Java
Translation Process Translates source code line by line at runtime Translates entire source code to machine code before execution
Error Reporting Errors displayed as program runs; partial output available even with errors Errors found at compile time before execution; no output if errors exist
Execution Speed Slower execution; each line translated every time it runs Faster execution; compiled machine code runs directly on CPU
Debugging Easier to test and debug interactively

with immediate feedback Harder to debug; all-or-nothing execution model
Key Difference: Python uses an interpreter, allowing line-by-line execution with immediate results, making it ideal for learning and interactive development.

Introduction to Python Programming
Why Python?
Python is a high-level, general-purpose programming language designed by Guido van Rossum and first released in 1991. Key advantages include:

Readability: Python syntax reads like English and closely mirrors natural thinking patterns
Ease of Learning: Fast to start with no complex setup required
Versatility: Used for web development, AI, data science, automation, and scientific research

Cross-Platform: Runs seamlessly on Windows, macOS, and Linux systems
Large Community: Millions of learners and professionals provide abundant resources and support
Google Colab as the IDE
Google Colab is a free, cloud-based IDE chosen for this course because it offers:

Zero Setup: No installation required; runs entirely in a web browser with only a Google account and internet access
Cell-by-Cell Execution: Code organized into cells that can be executed individually, with output displayed immediately below each cell; simplifies debugging by isolating errors to specific cells
AI Assistance: Integrated Google Gemini AI assistant that helps with code using plain English prompts

Pre-installed Libraries: Common libraries like Pandas, NumPy, and Matplotlib come pre-loaded; can be imported and used immediately
Free Hardware Access: Provides free access to powerful computing resources including CPUs, GPUs, and TPUs for machine learning and data science tasks
Easy Collaboration: Files saved directly to Google Drive; can be shared with others for real-time collaborative editing and feedback
Popular for ML and Data Science: Widely used for machine learning and data science projects with easy model export capabilities
The Computing to Programming Transition
The course follows a systematic approach:

Problem Statement: Define the problem to be solved
Algorithm: Create logic and instructions to solve the problem

Programming: Use Python to translate the algorithm into machine-readable code
Execution: The computer processes the code and produces output
Solution: The output solves the original problem
Computing provides the concepts (data, algorithms, logic), while programming provides the tools to implement these concepts. Python serves as the bridge between human logic and machine execution.

Lecture 8

Introduction to Python Programming Foundations
This lecture covers the fundamental concepts of Python programming, including setup in Google Colab, variable naming conventions, and string operations.

Google Colab Setup and Overview

Google Colab is a cloud-based Integrated Development Environment (IDE) accessible through Google search
To access: Search "Google Colab," click the first link, and open a new notebook
File extension for Python notebooks is IPYNB (IPython Notebook)
Code execution requires connecting to a runtime environment by clicking the run button (arrow icon)
File management options include:
Renaming notebooks
Locating files in Google Drive
Downloading files in IPYNB or PY format
Sharing directly with others via email
Uploading existing notebook files
Variables and Naming Conventions
What is a Variable?

Variables store values and allow you to

reference data without rewriting it
Example: message = "Hello World" stores the string in a variable that can be reused with print(message)
Variables can be reassigned multiple times, and Python tracks the current value
Naming Convention Rules:

Variable names can contain only letters, numbers, and underscores
Names must start with a letter or underscore, never a number
Spaces are not allowed; use underscores to separate words (e.g., greeting_message)
Avoid Python keywords and built-in function names (e.g., print, if, else, try, while, return)
Use short but descriptive names (e.g., student_name instead of s_n)
Be cautious with lowercase "L" and uppercase "O" as they resemble numbers 1 and 0

Use lowercase letters for variable names as a best practice
Error Handling:

Name Error occurs when a variable is misspelled or not defined before use
Python provides a traceback to help identify where errors occur
Consistency in spelling is critical from initialization to usage
Strings and String Operations
What is a String?

A string is a series of characters enclosed in single or double quotation marks
Any text within quotes is treated as a string in Python
Single and double quotes can be combined to allow quotes within strings (e.g., 'He said "Hello"')
String Methods and Functions:

Method Purpose Example
.title() Converts to title case "ada lovelace".title() → "Ada Lovelace"
.upper() Converts to uppercase "ada lovelace".upper() → "ADA LOVELACE"
.lower() Converts to lowercase "Ada Lovelace".lower() → "ada lovelace"
String Concatenation:

Combine strings using the + operator
Example: first_name + " " + last_name
Spaces must be enclosed in quotes as they are treated as strings
Whitespace Characters:

— Tab character (adds indentation)
— Newline character (starts a new line)
Example: print("Languages: Python JavaScript")
Stripping Whitespace:

.lstrip() — Removes whitespace from the left side of a string
.rstrip() — Removes whitespace from the right side of a string
.strip() — Removes whitespace from both sides of a string
Displaying Quotation Marks:

To display quotation marks in output, use alternating quote types
Example: 'He said "Hello"' will display the double quotes in output
Methods and Functions
Methods are actions Python performs on data, accessed using dot notation (e.g., variable.method())
Every method is followed by parentheses
Parentheses may contain arguments (additional information needed for the method to work)

The print() function displays output; methods like .title() may have empty parentheses if no additional information is needed

Key Takeaways

Google Colab provides an accessible platform for Python programming without local installation

Proper variable naming follows specific conventions and improves code readability

Strings are fundamental data types with numerous built-in methods for manipulation

Whitespace and special characters can be controlled for formatted output

Understanding error messages and tracebacks helps identify and fix programming mistakes

Lecture 9

Overview of Computing Models and Data Classification

This lecture builds upon the foundational IPO

(Input, Process, Output) model and explores critical concepts in computing, data science, and their real-world applications across various industries.

Key Topics Covered

1. IPO Model Foundation and Business Applications

Weather Forecasting Example:

Billions of live data points collected hourly from global networks
Applied models process inputs to generate forecasts
Demonstrates basic IPO model functionality
Uber Case Study:

Input Stage: Live GPS signals, rider location requests, local traffic data, weather conditions,

historical demand patterns
Process Stage: Matching algorithms determine optimal pricing and driver allocation
Output Stage: Driver pickup confirmation, estimated arrival time, final payment amount
2. Critical Risks in Algorithm Deployment
Knight Capital Trading Firm (2012):

A newly deployed autonomous trading algorithm contained a critical bug
In just 45 minutes, the algorithm executed 4 million trades automatically
Result: Company lost $440 million and went bankrupt
Root Cause: Algorithm was incorrectly coded and lacked adequate security measures and human oversight
Key Lesson: Correctness of transactions is more important than speed; proper testing and verification are essential before deployment

Implications for Business:

Speed of computing execution can amplify errors catastrophically
Without proper guardrails and verification, automation creates exponential risk
Process quality (algorithm correctness) is critical in the IPO model
3. Computing Integration in Non-Traditional Industries
Agriculture Sector:

AgroStar and other agro-tech firms use soil sensors, satellite imagery, and weather pattern data
Provide farmers with recommendations on what to plant, when to plant, and water usage
Recommendation engine functionality adapted for agricultural decisions
Healthcare:

ECG machines generate electrical activity data at 100 measurements per second
Input data includes heart parameters, arterial fibrillation indicators, blockage detection
Classification algorithms process data to output health diagnoses
Demonstrates clinical application of computing models
Fashion Retail - Zara Case Study:

Uses computing to determine which garments to produce and in what quantities
Collects global sales data across 24-hour periods from all store locations
Algorithms detect fast-selling and slow-selling designs
Competitive Advantage: Can plan production 6 months ahead
Critical Step: Process phase (algorithm analysis)

is most important for competitive advantage
Data Type: Time series data (sales trends tracked over time)

4. Four Pillars of Computing

All computing systems require these four foundational components:

Data: Raw material for processing
Algorithm: Processing logic and decision-making rules
Hardware: Physical infrastructure for computation
Software: Programs and applications
Removal of any pillar causes the system to fail.

5. DIKW Model - Data to Wisdom Pyramid

A hierarchical framework showing data transformation:

Data (Base Level):

Raw, unprocessed facts and figures
Numbers, coordinates, or observations without context
Example: "42" or "Mumbai"
Information (Second Level):

Data organized with context and structure
Answers "who, what, where, when" questions
Requires three elements: a question to answer, data processing, and relationship identification
Example: "Your age is 42" or "Your place of birth is Mumbai"
Knowledge (Third Level):

Accumulation of information over time
Pattern recognition and understanding relationships
Answers "why" questions

Allows confident action on information
Wisdom (Top Level):

Strategic decision-making application of knowledge
Considers uncertainty and risk
Addresses "what should we do" and "what is the cost"
Guides organizational actions
6. Netflix Case Study - DIKW Application
Data Layer: Raw user logs containing user ID, event type (pause/play), content ID, timestamp, and timecode

Information Layer: Aggregation reveals 3.2 million viewers paused content at 42 minutes 15 seconds, indicating a significant pattern

Knowledge Layer: Cross-referencing behavior with script analysis shows viewers paused due

to a confusing subplot lacking proper context; identifies root cause of user dropout

Wisdom Layer: Executive decision to edit production quality and add recap features to improve viewer retention

7. Bank Loan Approval Case Study - Ethical Considerations

The Problem with Misusing Information:

Bank data showed applicants from certain PIN codes defaulted 3.4 times more frequently
Incorrect interpretation: Deny loans to all applicants from those areas (redlining)
Ethical Issue: Geographic discrimination based on incomplete analysis
Correct Approach (Moving to Knowledge and Wisdom):

Investigate why defaults occur: lower income areas, informal employment, uneven income patterns
Include additional variables: income level, occupation, age, dependents
Build comprehensive models respecting regulatory requirements (RBI guidelines)
Make informed decisions that balance risk management with ethical lending
Key Learning: Operating only at information level without climbing to knowledge and wisdom leads to unethical and illegal practices

8. Data Classification Systems

Structured vs. Unstructured:

Type Format Storage Examples
Structured Rows and columns with defined data

types DBMS/RDBMS (MySQL, PostgreSQL, Oracle) Bank transactions, student marksheets, e-commerce orders
Unstructured No defined structure or fixed format Specialized databases PDFs, emails, images (JPG), audio (MP3), video (MP4), sensor streams
Semi-Structured Partial organization with key-value pairs NoSQL databases JSON files, XML files, email (EML) files
Business Implications:

Structured data: Lower processing cost, simpler analysis tools
Unstructured data: Higher processing cost, requires specialized skills (Natural Language Processing, computer vision)
Different techniques and expertise needed for each type
9. Time-Based Data Classification

Cross-Sectional Data:

Single survey or measurement point in time
Captures multiple variables at one time period
Example: Customer survey in June 2025 about phone brands, screen time, app usage
Limitation: Cannot show changes over time
Panel Data (Longitudinal Data):

Multiple measurements from same respondents over time
Tracks changes in multiple variables across time periods
Example: Monthly marks of students tracked over six months
Advantage: Can identify improvement, decline, or consistency patterns
Time Series Data:

Single variable measured at regular time intervals

Tracks one metric over time
Recent data has more predictive value than older data
Example: Stock price, solar power consumption, share prices tracked daily/monthly/yearly
Characteristic: Cannot use one time series variable to predict another time series variable
10. Quantitative vs. Qualitative Data
Quantitative (Quant) Data:

Numerical and measurable data
Uses measurement scales
Examples: height, weight, temperature, age
Qualitative (Qual) Data:

Descriptive and categorical data
Captures opinions, text, and non-numerical information
Examples: customer feedback, natural language text, survey responses

Summary of Key Takeaways

The IPO model is foundational but requires careful algorithm validation to prevent catastrophic business losses

Computing is revolutionizing traditional industries through intelligent data processing

The DIKW pyramid shows that data alone is insufficient; interpretation and decision-making require moving through information to knowledge and wisdom

Ethical application of computing requires comprehensive analysis and regulatory compliance

Data classification impacts processing costs, required expertise, and analytical techniques

Understanding data type (structured/semi-structured/unstructured) and temporal nature (cross-sectional/panel/time series) is essential for appropriate analysis

Lecture 10

Introduction to Python Programming: Variables, Numbers, Strings, Comments, and Lists

Overview

This lecture covers fundamental Python programming concepts including variable naming, string operations, numerical operations, type conversion, comments, and an introduction to lists. The session builds progressively from basic string manipulation to working with numbers and handling mixed data types.

String Operations and Variables

String Basics: Strings are series of characters written between single or double quotes in Python

Common String Operations:

Concatenation: combining multiple strings using the + operator

Special characters: for tabs and for new lines

Whitespace management: using strip(), lstrip(), and rstrip() methods to remove unwanted spaces
Displaying quotes: using backslashes to print quotation marks in output
Numerical Operations
Basic Arithmetic Operations:

Addition: 2 + 3 = 5
Subtraction: 3 - 2 = 1
Multiplication: 2 * 3 = 6 (using asterisk *)
Division: 3 / 2 = 1.5
Exponentiation: 3 ** 2 = 9 (using double asterisk **)
Order of Operations: Python follows the PEMDAS/BODMAS principle, executing operations in the correct mathematical order. Parentheses can override default order (e.g., (2 + 3) * 4 = 20)

Float Variables and Precision Issues

Float Definition: Any number with a decimal point is called a float (floating-point number)
Precision Anomaly: Python may display unexpected decimal places due to how computers internally represent numbers (e.g., 0.2 + 0.1 might display as 0.30000000000000004 instead of 0.3)
Important Note: This minor precision issue occurs in all programming languages and is generally of little concern for most applications
Combining Strings and Numbers
Type Error Problem: Attempting to concatenate strings directly with integers causes a TypeError because Python cannot combine string and integer types

Example: "happy" + 23 + "rd birthday" produces an error
Solution 1: Convert the number to a string by adding quotation marks (e.g., "23" instead of 23)

Limitation: Not ideal if the number is needed for calculations elsewhere in the program
Solution 2: Use the str() method to convert integers to strings

Example: "happy" + str(23) + "rd birthday" works correctly
Advantage: Preserves the original numeric variable for other uses
Comments in Python
Purpose: Comments allow programmers to write notes in plain English within code to explain the program's logic and approach
Syntax: Use the hash symbol # to indicate a comment; anything following # on a line is ignored by the Python interpreter
Professional Importance: Comments are essential for collaborative programming, helping team members understand code functionality

across different locations and projects
Best Practice: Write meaningful comments that describe what the code does and how it works
Exercise Tasks
Task 1: Write four arithmetic operations (addition, subtraction, multiplication, and division) that each result in the number 8, enclosed in print statements

Task 2: Store a favorite number in a variable and create a message that reveals the favorite number, then print that message

Task 3: Choose two previously written programs and add at least one comment to each describing what the program does

Introduction to Lists
Definition: Lists are Python's data structures that store sets of information in one place, from a

few items to millions of items
Significance: Lists are one of Python's most powerful features and are accessible to new programmers
Importance: They tie together many important programming concepts that will be explored further in upcoming lectures

Lecture 11

Introduction to Lists in Python
A list is a collection of items arranged in a particular order. Lists can contain any type of elements, such as letters, numbers, or names, and the items do not need to be related to each other in any way.

Overview of Lists
Lists are fundamental data structures in Python that allow you to store multiple values in a single variable. The naming convention for lists

typically uses plural forms (e.g., letters, digits, names, motorcycles) since they contain multiple elements.

Creating and Printing Lists
Syntax: Lists are created using square brackets with elements separated by commas
Example: bicycles = ['Trek', 'Cannondale', 'Red Line', 'Specialized']
Output: Printing a list displays all its elements in the same format they were stored
print(bicycles) outputs: ['Trek', 'Cannondale', 'Red Line', 'Specialized']
Accessing Elements in a List
Index Positions: Lists are ordered collections where each element has a specific position
The first element is at index 0 (not 1)
The second element is at index 1
The third element is at index 2, and so forth
Syntax: Access elements using the list name

followed by the index in square brackets
bicycles[0] returns 'Trek'
bicycles[1] returns 'Cannondale'
Formatting Elements: You can apply string methods to accessed elements
bicycles[0].title() converts 'trek' to 'Trek'
.upper() converts to uppercase; .lower() converts to lowercase
Accessing the Last Element
Negative Indexing: Python provides a special syntax for accessing elements from the end of the list
bicycles[-1] always returns the last element (in this case, 'Specialized')
bicycles[-2] returns the second-to-last element
bicycles[-3] returns the third-to-last element, and so on
Benefit: This method is useful when you don't know the list's length or when working with dynamic lists

Using Individual List Values

String Concatenation: You can combine list values with strings using the concatenation operator (+)

Example: message = 'My first bicycle was a ' + bicycles[0].title() + '.'

Output: 'My first bicycle was a Trek.'

Modifying List Elements

Changing Values: To modify an element, use the list name, index, and assignment operator

Syntax: list_name[index] = new_value

Example: motorcycles[0] = 'Ducati' replaces 'Honda' with 'Ducati'

Effect: The new value replaces the old value at that specific position

Adding Elements to a List

Appending Elements

Method: The append() method adds a new element to the end of the list

Syntax: list_name.append(new_value)

Example: motorcycles.append('Ducati') adds 'Ducati' at the end

Building Dynamic Lists: You can start with an empty list and add elements one by one

```
motorcycles = []
motorcycles.append('Honda')
motorcycles.append('Yamaha')
motorcycles.append('Suzuki')
```

Inserting Elements at Specific Positions

Method: The insert() method adds an element at a specified position

Syntax: list_name.insert(index, new_value)

Example: motorcycles.insert(0, 'Ducati') places 'Ducati' at the beginning (index 0)

Example: motorcycles.insert(1, 'Ducati') places 'Ducati' at the second position

Effect: All elements at and after the specified index shift one position to the right

Removing Elements from a List

Using the del Statement

Purpose: Permanently deletes an element from a list when you know its position
Syntax: del list_name[index]
Example: del motorcycles[0] removes the first element
Characteristics: Once deleted, the value cannot be retrieved
Using the pop() Method
Purpose: Removes an element from the list while retaining access to that value
Syntax: list_name.pop() (removes the last element) or list_name.pop(index) (removes specific position)
Example: popped_motorcycle = motorcycles.pop() removes and stores the last element
Default Behavior: Without an index, pop() always removes the last element
Stack Concept: Think of a list as a stack of plates where pop() removes from the top (end)

Practical Use: If you need to use the removed value elsewhere in your code, use pop()
Example: first_owned = motorcycles.pop(0) removes and stores the first element
Using the remove() Method
Purpose: Removes an element when you know its value but not its position
Syntax: list_name.remove(value)
Example: motorcycles.remove('Ducati') removes 'Ducati' from the list
Argument Options: You can pass the value directly or use a variable containing the value
Direct: motorcycles.remove('Ducati')
Variable: expensive = 'Ducati' then motorcycles.remove(expensive)
Important: The remove() method only deletes the first occurrence of a value
If a value appears multiple times, only the first instance is removed
Use loops to remove all occurrences of a

repeated value

Choosing the Right Deletion Method

Use del when you want to permanently remove an element and have no further use for it

Use pop() when you need to remove an element but want to use its value elsewhere in your code

Use remove() when you know the value to remove but not its position in the list

Practice Exercises

Exercise 1: Create a guest list for dinner. Make a list with at least three people you would invite (living or deceased), then use the list to print personalized invitation messages to each person.

Exercise 2: Modify the program from Exercise 1. Add a statement announcing which guest cannot make it. Replace that guest's name with a new invitee and print a new set of invitation messages.

Exercise 3: Expand your dinner table. Add a statement indicating you found a bigger table. Use insert() to add one guest to the middle of your list, use append() to add one guest to the end, and print updated invitation messages for all guests.

Exercise 4: Reduce guest list. Add a message stating only two guests can be invited. Use the pop() method to remove guests one by one, and use the del statement to remove the remaining two guests. Print messages confirming which guests are still invited.

Lecture 12

Organizing a List

This lecture covers various operations for

organizing and managing lists in Python, including sorting, reversing, and finding list length, along with methods to avoid common errors.

Sorting a List Permanently with the Sort Method
The sort() method arranges list elements in alphabetical, numerical, or reverse order permanently
Syntax: list_name.sort()
Example: A list of car names (BMW, Audi, Toyota, Subaru) can be sorted alphabetically using cars.sort(), resulting in (Audi, BMW, Subaru, Toyota)
To sort in reverse order, use: cars.sort(reverse=True), which produces (Toyota, Subaru, BMW, Audi)
The sort method permanently changes the list order for the rest of the program
Sorting a List Temporarily with the Sorted

Function

The sorted() function displays a list in sorted order without permanently changing the original list

Syntax: print(sorted(list_name))

Example: sorted(cars) displays (Audi, BMW, Subaru, Toyota), but when you print the original list again, it remains in its original order (BMW, Audi, Toyota, Subaru)

The sorted function accepts the same reverse=True argument for reverse sorting

Example: sorted_cars = sorted(cars, reverse=True) stores the reverse-sorted list in a variable temporarily

Printing a List in Reverse Order

The reverse() method permanently reverses the order of a list

Syntax: list_name.reverse()

Example: cars.reverse() changes (BMW, Audi, Toyota, Subaru) to (Subaru, Toyota, Audi, BMW)

Unlike sorting, reversing simply inverts the order without any alphabetical arrangement
The change is permanent, but you can apply reverse() again to restore the original order
Finding the Length of a List
The len() function returns the number of items in a list
Syntax: len(list_name)
Example: A list with four cars has a length of 4
Important: Do not confuse length with index position (a list of length 4 has indices 0, 1, 2, and 3)
Avoiding Index Errors When Working with Lists
An index error occurs when trying to access an element at an index that does not exist
Error message: "list index out of range"
Common cause: The "off-by-one" nature of indexing—lists start at index 0, so a list of length 3 has indices 0, 1, and 2, not 1, 2, and 3
Example: Trying to access motorcycles[3] when

the list only has three items (indices 0-2) causes an index error
Negative indexing can be used to access elements from the end:
list_name[-1] accesses the last item
list_name[-2] accesses the second-to-last item
Empty lists will produce an index error if you attempt to access any element
To fix index errors, verify that the requested index exists within the valid range for the list
Practice Exercises
Exercise 1: Organizing Places to Visit

Store at least five desired travel locations in a list (not in alphabetical order)
Practice printing the list in its original order
Use sorted() to display in alphabetical order while maintaining original order
Use sorted() with reverse=True for reverse alphabetical order

Use reverse() to permanently change the list order
Apply reverse() again to restore the original order
Use sort() to permanently arrange the list in alphabetical order
Exercise 2: Applying All Functions

Create a list of items (mountains, rivers, countries, or languages)
Use each function introduced in this chapter (sort, sorted, reverse, len) at least once
Exercise 3: Creating Index Errors

Intentionally create an index error by accessing an invalid index (e.g., motorcycles[5] on a three-item list)
Practice identifying and correcting the error

Lecture 13
Working with Lists: Looping and Indentation

Management

Overview

This lecture covers foundational concepts for working with lists in Python, specifically focusing on how to iterate through list elements using loops and how to avoid common indentation errors that can occur during this process.

Looping Through an Entire List

Introduction to For Loops

Purpose: For loops allow iteration through all elements in a list without writing repetitive code

Problem Solved: Eliminates the need to write individual print statements for each list element, which becomes impractical with large lists

Benefit: Code automatically adapts if list length changes

For Loop Syntax

for [variable_name] in [list_name]:

[statement to execute]
for: keyword indicating loop start
variable_name: stores each individual value from the list during each iteration
in: keyword connecting variable to list
list_name: the list to iterate through
: (colon): required syntax element that marks the beginning of the loop body
How For Loops Work
Python takes the first value from the list and stores it in the variable
Executes all indented statements below the for line
Moves to the next value and repeats the process
Loop terminates automatically when all list elements are processed
Multiple Statements in Loops
You can include multiple statements within a loop
All indented lines following the for statement are

executed for each list element
Non-indented lines after the loop execute only once, after the loop completes
Any number of Python statements can be included in a loop
Example: Magician Messages
Simple output: Print each name from a list
Message output: Combine list values with strings using concatenation (+)
String methods like .title() can format output within loops
Avoiding Indentation Errors
Understanding Indentation's Role
Python uses indentation (whitespace) to determine code structure
Indented lines following a for loop are considered part of that loop
Proper indentation is essential for correct program execution
Indentation makes code visually organized and

readable

Common Indentation Errors

1. Missing Indentation After For Statement
   Error Type: Syntax Error - "Expected an indented block"
   Cause: No indentation provided for statement following for loop
   Solution: Press Tab or add spaces to indent the statement
   Impact: Program will not run

2. Forgetting to Indent Additional Lines
   Error Type: Logical Error (no syntax error reported)
   Cause: Some statements meant for the loop are not indented
   Result: Unindented statements execute only once after loop completes, using only the last value from the loop variable
   Solution: Indent all statements that should be

part of the loop
Impact: Program runs but produces incorrect output

3. Unnecessary Indentation
   Error Type: Syntax Error - "Unexpected indent"
   Cause: Indenting lines that should not be part of any block
   Solution: Remove extra whitespace from the beginning of the line
   Impact: Program will not run

4. Missing Colon After For Statement
   Error Type: Syntax Error - "Expected colon"
   Cause: Forgetting the : at the end of the for statement
   Solution: Add a colon at the end of the for line
   Impact: Program will not run
   Key Principle
   The scope and execution context of a for loop is determined entirely by indentation levels.

Any indented line following a for statement is part of that loop; non-indented lines that follow are outside the loop.

Practical Application
Store related items in a list
Use for loops to process each item
Add personalized messages or perform operations on each element
Control which statements execute repeatedly versus once using indentation
Verify syntax (colons, indentation) before running code

Lecture 14

Overview of Computing Fundamentals and Python Programming Introduction
This lecture covered essential computing concepts and introduced Python programming, building on previous discussions about data

types and classifications.

Data Classification
Quantitative vs. Qualitative Data
Quantitative (Quant) Data:

Numerical data that can be measured using a predefined scale
Examples: marks in a test, height in centimeters, weight in kilograms
Can be further classified as:
Discrete: Countable whole numbers (e.g., number of students = 102)
Continuous: Values within a range (e.g., height 100-190 cm, temperature 0-36.8°C)
Mathematical operations (addition, subtraction, mean, median) can be performed on quantitative data
Qualitative (Qual) Data:

Data generated from perception, feeling, and thinking with no absolute measurement scale
Examples: blood group (A positive, B positive), place of birth, movie ratings (1-5 stars), customer satisfaction (poor, fair, good, excellent)
Categorical in nature; names and opinions in natural language
Only the mode (frequency of occurrence) can be calculated
No mathematical operations possible without conversion
Context Matters
Square feet of a house and price are continuous quantitative data because they fall within specific ranges, not all values are possible
Data classification depends on context, not just the numbers themselves
Primary vs. Secondary Data
Primary Data:

Collected by yourself or your team for a specific analysis purpose
Collection methods: surveys, Google Forms, sensors, interviews
Advantages: tailored to exact research questions, full control over data collection and quality
Disadvantages: time-consuming, costly, results in small sample sizes
Secondary Data:

Already collected and available from external sources
Examples: World Bank data, IMF data, NSSO data, Wikipedia, stock exchange data
Advantages: large scale data, often free or low-cost, readily available
Disadvantages: may not perfectly match research questions, quality depends on the collector's credibility

Credibility of the source is crucial (e.g., World Bank data is more reliable than unknown private sources)
Metadata:

Data about data (e.g., library catalog information about books rather than book content itself)
Quiz 1 Details
Scheduled for next Saturday at 8:00 AM
Duration: 20 minutes
Format: 60 MCQ questions (approximately 20 seconds per question)
Syllabus: All recorded lectures and live sessions up to September 20th, including "Working with Lists Part 2"
Best of three quiz scores will be considered for final evaluation
Algorithm Fundamentals
Definition: A logical sequence of steps for solving a problem; the intellectual foundation of

computing.

Essential Properties:

Finiteness: Algorithm must terminate after a finite number of steps

Good example: Even/Odd number checker (uses modulus operator)
Bad example: Dividing a number by 2 until reaching exactly zero (infinite loop)
Definiteness: Output must be definite and unambiguous

Every step must be precisely defined
No room for interpretation
Same input produces same output every time
Example: Adding ingredients in a specific order for making tea or bread
Input: Can be zero or more inputs that define

what the algorithm operates on

Output: Must produce at least one output; an algorithm without output is pointless

Effectiveness: Algorithm should not use infinite resources

Resource optimization is critical for computational efficiency
Energy consumption and computational power must be minimized
Representation: Flowcharts visually represent algorithms with start, stop, decision points, and processes.

Computing Pillars
Hardware
Physical components: keyboard, mouse, monitor, screen, sensors, CPU, GPU

GPU (Graphical Processing Unit): Multiple weaker cores working in parallel for simple, parallelizable tasks
CPU (Central Processing Unit): Few but stronger cores for complex computations
Example: Mac Mini M4 - compact desktop combining CPU and GPU cores, highly efficient for moderate AI models
Memory Types:
RAM: Volatile, fast, temporary storage for variables and running programs
ROM: Non-volatile, contains hard-coded bootloader instructions
Secondary Storage (SSD, HDD, USB drives): Slow but large capacity storage
Software
Operating System (OS): Master program (Windows, Mac OS, Linux, Android, iOS, ChromeOS) that manages processes, memory, files, devices, security, and user interface

Utility Software: File management, disk cleanup, defragmentation, antivirus, compression tools (7Zip, WinRAR)
Application Software:
Communication: Email, instant messaging (WhatsApp, Telegram, Slack)
Video conferencing: Zoom, Google Meet
Web browsers: Chrome, Firefox, Safari
Entertainment: Games, media players, streaming platforms (Netflix, Prime Video, YouTube)
Professional: Jupyter Notebook, Tableau, AutoCAD, Photoshop, MATLAB
Programming Languages and IDEs
Interpreter vs. Compiler:

Interpreter (Python): Translates code line by line at runtime; shows errors progressively
Compiler (C, C++): Translates entire source code before execution; no output until all errors are fixed

Python Advantages:

Designed by Guido van Rossum in 1991
High-level, general-purpose language
Simple, easy to understand and write
Universal: cross-platform compatibility
Large community support
Extensive libraries and inbuilt functions
Google Colab:

Cloud-based IDE requiring no setup
Free access to powerful hardware (GPU and TPU)
Popular in ML and data science
Integrated development environment with compilation and execution in one place
Python Basics
Print Function:

print("Hello World") - directly prints string

message = "Hello World" then print(message) - prints variable content
Variables store data and can be reassigned (value gets overwritten)
Variable names must match exactly; case-sensitive and spelling matters
String Methods:

Methods are called using dot notation: variable_name.method_name()
title() method: Converts string to title case (first letter of each word capitalized)
Example: "ada lovelace".title() returns "Ada Lovelace"
Nesting functions: Multiple functions can be combined, e.g., print(name.title())
Final Examination:

Will be offline (pen and paper format)
Will include Python code writing questions

Students must practice writing code manually

Lecture 15

Foundations of Python Programming: Working with Lists

Overview

This lecture covers three major topics in Python list manipulation: making numerical lists, list comprehensions, and working with parts of a list (slicing). The session builds on previously covered concepts including looping through lists and avoiding indentation errors.

Making Numerical Lists

The Range Function

Purpose: Generates a series of numbers efficiently in Python

Syntax: for value in range(start, end):

Key Characteristic: The end value is not included

in the output. For example, range(1, 5) produces 1, 2, 3, 4 (not 5)

To include the end value: Add 1 to the end parameter. For range(1, 6) produces 1, 2, 3, 4, 5

Converting Range to a List

Use the list() function to convert a range into a Python list

Example: numbers = list(range(1, 6)) creates [1, 2, 3, 4, 5]

Once converted to a list, all standard list operations (append, concatenation, etc.) can be applied

Skipping Numbers in a Range

Use the third argument in range to skip values

Syntax: range(start, end, step)

Example: even_numbers = list(range(2, 11, 2)) produces [2, 4, 6, 8, 10]

The step value determines how many numbers to skip between each printed value

Creating Lists of Squares

Without list comprehension (4 lines):

```
squares = []
for value in range(1, 11):
squares.append(value ** 2)
```

More concise approach (2 lines):

```
squares = []
for value in range(1, 11):
squares.append(value ** 2)
```

The temporary variable can be removed by passing the expression directly to append()

Simple Statistics with Numerical Lists

min(): Returns the smallest value in a list

max(): Returns the largest value in a list

sum(): Adds all numerical values in the list

Example: For list digits = [1, 2, 3, 4, 5, 6, 7, 8, 9, 0]

min(digits) returns 0

max(digits) returns 9

sum(digits) returns 45

List Comprehensions

What is a List Comprehension?

A concise, single-line method to create numerical lists

Combines a for loop and list creation into one statement

Automatically appends each new element to the list

Syntax

list_name = [expression for variable in range()]

Example

Traditional approach (4 lines):

```
squares = []
for value in range(1, 11):
squares.append(value ** 2)
```

List comprehension (1 line):

```
squares = [value ** 2 for value in range(1, 11)]
```

Important Note

No colon is used at the end of the for statement in list comprehensions

This differs from traditional for loops, which require a colon

The expression (value ** 2) comes before the for statement

Working with Parts of a List (Slicing)

What is a Slice?

A subset or portion of a list

Accessed using the syntax: list_name[start:end]

Slicing Syntax and Rules

Start index: Inclusive (included in the slice)

End index: Exclusive (not included in the slice)

Similar to the range() function behavior

Example: players[0:3] returns elements at indices 0, 1, 2 (not 3)

Omitting Indices

Omit start index: players[:3] automatically starts from index 0

Omit end index: players[2:] goes from index 2 to the end of the list

Omit both: players[:] returns the entire list

Negative Indices in Slices

Negative indices count from the end of the list

players[-1] returns the last element

players[-2] returns the second-to-last element

Example: players[-3:] returns the last three players in the list

Looping Through a Slice

Use a for loop to iterate through only a portion of a list

Example:

```
for player in players[:3]:
print(player.title())
```

This loops through only the first three players and prints them in title case

Practical Applications

Processing data in chunks of specific sizes

Displaying information in pages with controlled amounts of data per page

Accessing subsets of data without modifying the original list

Exercises to Practice

Use a for loop to print numbers from 1 to 20

Create a list from 1 to 1 million and use min(), max(), and sum() functions

Use the third argument of range() to create a list of odd numbers from 1 to 20

Create a list of multiples of 3 from 3 to 30

Generate a list of the first 10 cubes using both traditional loops and list comprehensions

Practice slicing operations with various start and end indices

Combine slicing with for loops to process subsets of data

Key Takeaways

The range() function excludes its end value

List comprehensions provide a concise, Pythonic way to create lists

Slicing allows efficient access to list subsets without modifying the original list

Negative indices provide convenient access

from the end of a list
These tools are essential for data processing and web application development

Lecture 16

Foundations of Computing: Working with Lists and Tuples

Overview
This lecture covers advanced list manipulation techniques and introduces tuples as an immutable data structure. The session focuses on copying lists properly, understanding the difference between creating references and true copies, and working with tuples for data that should not change.

Copying a List
Creating a True Copy Using Slicing:

To create an independent copy of a list, use the slice notation with empty indices: new_list = original_list[:]
This syntax tells Python to create a slice starting from the first item and ending with the last item, producing a complete copy
The copied list is a separate entity, not just another reference to the same list
Example - Copying Food Lists:

Original list my_foods contains: pizza, falafel, carrot cake
Using friend_foods = my_foods[:] creates an independent copy
Changes to one list do not affect the other list
When canoli is added to my_foods and ice cream is added to friend_foods, each list maintains only its own items
Incorrect Method - Creating References Instead of Copies

Assignment Without Slicing:

Using friend_foods = my_foods (without slices brackets) does not create a copy
Instead, it creates a second variable name pointing to the same list
Any modifications through either variable name affect the same underlying list
Both variables are aliases for the identical list object
Example - Reference Problem:

When canoli is appended to my_foods, it also appears in friend_foods
When ice cream is appended to friend_foods, it also appears in my_foods
This proves both variables reference the same list, not separate lists
Tuples - Immutable Data Structures
Definition and Purpose:

A tuple is an immutable list where values cannot be changed, removed, or added after creation
Use parentheses () instead of square brackets [] to define tuples
Tuples are useful for data that should remain constant throughout a program's execution
Examples: storing rectangle dimensions, configuration values, or other fixed data
Syntax:

List definition: my_list = [item1, item2, item3]
Tuple definition: my_tuple = (item1, item2, item3)
Accessing Tuple Elements:

Individual elements are accessed using index notation, just like lists: dimensions[0]
Tuples can be iterated using for loops: for item in my_tuple:
All read operations work identically to lists

Attempting Modifications - TypeError:

Trying to modify a tuple element raises a TypeError: "tuple object does not support item assignment"
Example: dimensions[0] = 250 will fail if dimensions is a tuple
This error protection prevents accidental modifications to critical data
Overwriting Entire Tuples:

While individual values cannot be changed, the entire tuple can be replaced with a new one
Example: dimensions = (400, 100) replaces the old tuple completely
This is allowed because it reassigns the variable, not modifying the existing tuple
Practical Exercises
Exercise 1 - List Slicing:

Create a list of nine cars
Print the first three items using slice: cars[:3]
Print the middle three items using slice: cars[3:6]
Print the last three items using slice: cars[6:]
Exercise 2 - Copying and Modifying Lists:

Create a list of at least three favorite pizzas
Make a copy using: friend_pizzas = my_pizzas[:]
Add different pizzas to each list to prove they are separate
Use for loops to print each list with appropriate messages
Exercise 3 - Working with Tuples:

Store five basic foods in a tuple
Use a for loop to print each food
Attempt to modify one item to demonstrate the TypeError
Create a new tuple with modified items and display the revised menu

Key Takeaways

List Slicing for Copying: Always use list[:] to create independent copies; avoid simple assignment which creates references

Tuple Immutability: Tuples provide data protection by preventing accidental modifications

Proper Data Structure Selection: Choose lists for dynamic data that changes; choose tuples for fixed, unchanging data

Python's Indentation: Proper indentation is critical for program structure and avoiding errors

Iteration Methods: Both lists and tuples can be effectively traversed using for loops with identical syntax

Lecture 17

Introduction to Conditional Tests in Foundations of Computing

This lecture introduces conditional tests, the

foundational concept for if statements in Python programming. Dr. Aman Pathak discusses how to evaluate conditions that return true or false values, which form the basis for controlling program flow.

Overview of If Statements Chapter
The course is transitioning from working with lists to exploring if statements. The chapter covers three major topics:

Conditional tests
If statements
Using if statements with lists
This session focuses exclusively on conditional tests and their various applications.

Practical Introduction: Car Names Example
Scenario: Display car names in title case, except BMW which should appear in all uppercase.

Code Structure:

Uses a for loop to iterate through a list of cars
Implements an if statement to check if the car is BMW
If condition is true: print in uppercase using .upper() method
If condition is false (else): print in title case using .title() method
Key Concept: The indentation after the if and else keywords indicates which code block executes based on the conditional test result.

Conditional Tests: Core Definition
Definition: An expression evaluated as true or false at the heart of every if statement
Purpose: Python uses true and false values to decide whether to execute code following an if statement

Execution Logic:
If test evaluates to true → execute the code following the if statement
If test evaluates to false → ignore the code following the if statement
Checking for Equality
Single vs. Double Equal Signs:

Single equal sign (=): Assignment operator that assigns a value to a variable
Double equal sign (==): Comparison operator that checks if two values are equal
Example:

```
car = "BMW" # Assignment
car == "BMW" # Comparison (returns True)
car == "Audi" # Comparison (returns False)
```

Result: The equality operator returns true if values match on both sides; otherwise returns false.

Ignoring Case When Checking for Equality
Case Sensitivity Issue:

Python treats uppercase and lowercase letters differently
"Audi" ≠ "audi" in Python's comparison
Two values with different capitalization are not considered equal
Solution: Using the .lower() Method

Convert variable values to lowercase before comparison
The .lower() method does not permanently change the stored value
Only affects the comparison operation
Example:

```
car = "Audi"
car.lower() == "audi" # Returns True
```

print(car) # Still prints "Audi" (unchanged)
Real-World Application: Websites use this technique to ensure unique usernames regardless of capitalization, preventing "John," "JOHN," and "john" from being treated as different usernames.

Checking for Inequality
Inequality Operator: != (exclamation point + equal sign)

The exclamation point represents "not"
Tests whether two values are not equal
Example:

requested_topping = "mushrooms"
if requested_topping != "anchovies":
print("Mushrooms are not anchovies") # Executes (returns True)
Logic:

If values do not match → returns true

If values match → returns false

Numerical Comparisons

Comparison Operators:

== : Equal to

!= : Not equal to

< : Less than

<= : Less than or equal to

: Greater than

= : Greater than or equal to

Example:

age = 19

age < 21 # Returns True

age >= 21 # Returns False

age == 19 # Returns True

Using "and" to Check Multiple Conditions

Keyword: and combines two conditional tests

Logic:

If both tests pass → overall expression evaluates to true
If either test fails or both fail → overall expression evaluates to false
Example:

```
age_0 = 22
age_1 = 18
if (age_0 >= 21) and (age_1 >= 21):
print("Both are old enough") # Does not execute
```

(False and False = False)
Best Practice: Use parentheses to separate individual conditions for improved readability.

Using "or" to Check Multiple Conditions
Keyword: or allows checking multiple conditions with different logic

Logic:

Passes when either or both individual tests pass
Fails only when both individual tests fail
Example:

```
age_0 = 22
age_1 = 18
if (age_0 >= 21) or (age_1 >= 21):
print("At least one is old enough") # Executes
```

(True or False = True)
Comparison:

and: Requires ALL conditions to be true
or: Requires AT LEAST ONE condition to be true
Checking Whether a Value is in a List
Keyword: in checks if a value exists within a list

Use Cases:

Verify if a username exists in current usernames
Check if a location exists in a known locations list
Validate pizza toppings against available options
Example:

requested_toppings = ["mushrooms", "onions", "pineapple"]
"mushrooms" in requested_toppings # Returns True
"pepperoni" in requested_toppings # Returns False
Checking Whether a Value is Not in a List
Keywords: not in checks if a value is absent from a list

Practical Application: Verify whether a user has been banned before allowing them to post comments on a forum

Example:

```
banned_users = ["Andrew", "Carolina", "David"]
user = "Mary"
if user not in banned_users:
print("Mary, you can post a response") #
```

Executes (True)

Difference from in:

in → Checks presence

not in → Checks absence

Boolean Expressions

Definition: A Boolean expression is another name for a conditional test that returns either true or false

Boolean Values:

True and False are keywords in Python

Used to track conditions such as whether a game is active or if a user can edit content
Example:

game_active = True
user_can_edit = False

if game_active:
print("Game is running") # Executes (condition is True)

if user_can_edit:
print("You can edit") # Does not execute (condition is False)
Purpose: Provides an efficient way to track the state of a program or important conditions.

Practice Exercises
Students are assigned to complete exercises focusing on:

Create 10 Conditional Tests:

At least 5 tests that evaluate to true
At least 5 tests that evaluate to false
Coverage Requirements:

Test equality and inequality with strings
Test using the .lower() function
Numerical tests with comparison operators (==, !=, <, >, <=, >=)
Test using and and or keywords
Test whether items are in a list
Test whether items are not in a list
Format: Print a statement describing each test and predict its result before running the code.

Lecture 18

Foundations of Computing: If Statements

Overview

This lecture covers the practical implementation of if statements in Python, focusing on how to use conditional logic to control program flow. The session builds on previously discussed conditional tests and introduces various structures for making decisions in code: simple if statements, if-else chains, if-elif-else chains with multiple conditions, and multiple independent if statements.

Simple If Statements

Basic Structure: An if statement consists of the if keyword, followed by a conditional test that evaluates to True or False, and then the code block to execute if the condition is true

Execution Flow: If the conditional test evaluates to True, Python executes the code following the if statement. If it evaluates to False, Python skips the block and looks for an else block

Indentation: Proper indentation is critical in Python. All code within an if block must be indented from the if keyword. Code at the same indentation level as if is outside the block
Multiple Statements: An if block can contain multiple lines of code, as long as they are properly indented
Example: if age >= 18: followed by indented print statements will execute only when the condition is true
If-Else Statements
Purpose: If-else statements allow you to execute one action when a condition is true and a different action when the condition is false
Structure: The else block executes only when all preceding if/elif conditions return False
Advantages: Instead of having no output when a condition fails, the else block provides an alternative code path
Example: If a person's age is less than 18, one

message prints; otherwise, a different message about being old enough to vote prints
Execution: Python checks the if condition first. Only if it's false does Python check for and execute the else block
If-Elif-Else Chains
Multiple Conditions: Used when you need to test more than two possible situations
Sequential Evaluation: Python executes only one block in an if-elif-else chain, testing conditions in order until one passes
Efficiency: Once a condition passes, Python executes that block's code and skips all remaining tests in the chain
Structure: Can have one if statement, multiple elif statements, and optionally one else statement
Example Use Case: An amusement park with different admission prices based on age groups (under 4 free, 4-18 costs

5
,
18
+
c
o
s
t
s
5,18+costs10, 65+ costs $3)
Multiple Elif Blocks
Flexibility: You can use as many elif blocks as needed to check multiple conditions
Code Optimization: Instead of writing separate print statements in each block, assign values to a variable within each condition, then use a single print statement after the entire chain
Benefits: This approach makes code more concise and easier to maintain—changes to output messages only require updating one print

statement
Example: Check if age is under 4, then elif under 18, then elif under 65, then else for seniors—each assigns a different price value, followed by one print statement
Omitting the Else Block
Optional Else: Python does not require an else block at the end of an if-elif chain
Use Case: Sometimes it is clearer and more appropriate to use only if and elif statements without a catch-all else block
Advantage: Prevents executing code for invalid or unexpected data that doesn't match any specific condition
Example: For age validation, you might have if statements checking for specific age ranges without an else block, allowing the program to do nothing if no condition matches
Testing Multiple Conditions with Independent If Statements

When to Use: Use multiple independent if statements when you need to check all conditions and execute code for every condition that is true, not just the first one
Difference from If-Elif-Else: In an if-elif-else chain, Python stops after the first true condition. With independent if statements, all conditions are checked
Syntax: Each if statement is completely separate with no elif or else; each can execute independently of the others
Real-World Example: A pizzeria program checking for multiple toppings—if using if-elif-else, only the first topping would be added; with multiple if statements, all requested toppings are added
Code Structure:
If-elif-else is appropriate when only one code block should run
Multiple if statements are appropriate when

multiple code blocks might need to run

Key Concepts and Best Practices

Indentation Matters: Python uses indentation to determine which statements belong to which block

Logical Testing: Use comparison operators (==, !=, >, <, >=, <=) and logical operators (and, or, not) to create conditions

Sequential Processing: Python always processes code from top to bottom, checking conditions in the order they appear

Efficiency vs. Completeness: Choose between if-elif-else chains (when only one condition should trigger) and multiple if statements (when all matching conditions should trigger)

Practice Exercises

Students are assigned five exercises to practice these concepts:

Simple If Statement: Create a variable for alien

color and test if it's green using an if statement
If-Else Chain: Expand the alien example to include an else block for non-green aliens
If-Elif-Else Chain: Create a three-condition chain for green (5 points), yellow (10 points), and red (15 points) aliens
Life Stage Determination: Write an if-elif-else chain using age to determine life stage (baby, toddler, kid, teenager, adult, elder)
Multiple Independent If Statements: Create a list of favorite fruits and use five independent if statements to check if specific fruits are in the list

Lecture 19

Lecture Overview: Introduction to Python Programming Fundamentals

This lecture covers essential Python programming concepts with emphasis on

understanding fundamental knowledge rather than relying solely on AI-generated code. The session discusses practical coding topics including string operations, numeric computations, list manipulation, and best practices for programming literacy.

Why Learning Programming Fundamentals Matters
The Role of LLMs in Modern Development

LLMs (Large Language Models) can generate code from plain English prompts, but they are not a replacement for understanding programming basics
Using LLMs effectively requires foundational knowledge to:
Provide meaningful prompts to the AI
Evaluate whether the generated code is correct and efficient

Debug and modify code for specific contexts
Identify and fix errors in the output
Analogy: English Language Literacy

Just as children must learn the alphabet (ABCD) to write essays, even with AI assistance, programmers need to understand programming fundamentals
Without basic knowledge, it becomes impossible to:
Evaluate code quality
Detect logic errors ("elvish language" problem)
Make targeted modifications to existing code
Read and understand complex code written by others
String Operations and Formatting
Basic String Manipulation

Strings are text values stored within single or double quotation marks

Variables can store string values: name = "Alice"
Print function displays string values: print(name)
String Methods

Method Function Example
.title() Converts to title case (first letter of each word capitalized) "alice lovelace".title() → "Alice Lovelace"
.upper() Converts all characters to uppercase "alice".upper() → "ALICE"
.lower() Converts all characters to lowercase "ALICE".lower() → "alice"
.strip() Removes whitespace from both ends " alice ".strip() → "alice"
.lstrip() Removes whitespace from left side only " alice ".lstrip() → "alice "
.rstrip() Removes whitespace from right side only " alice ".rstrip() → " alice"
String Concatenation

Use the + operator to join strings together

Example: first_name + " " + last_name

Whitespace is controlled by adding space strings: " "

Whitespace Characters

(backslash t): Inserts a tab space

(backslash n): Inserts a new line

Working with Numbers

Basic Arithmetic Operations

Operator Operation Example

- Addition 2 + 3 = 5
- Subtraction 3 - 2 = 1
- Multiplication 2 * 3 = 6

/ Division 6 / 2 = 3

** Exponentiation (power) 3 ** 2 = 9

Order of Operations (BODMAS)

Brackets have highest precedence
Then exponents
Then multiplication and division (left to right)
Finally addition and subtraction (left to right)
Integers vs. Floats

Integers: Whole numbers without decimal points (2, 3, 10)
Floats: Numbers with decimal points (0.1, 0.2, 3.14)
Floating-Point Precision Issues

Operations with decimal numbers can produce unexpected precision variations
Example: 0.2 + 0.1 may result in 0.30000000000000004 instead of 0.3
This occurs due to internal binary representation limitations
Formatting Decimal Output

Use string formatting to limit decimal places: .Xf where X is the number of decimal places
Syntax: print(f"{variable:.3f}")
Example: 0.005 * 1.009 displayed as .3f shows 0.005 instead of full precision
Type Conversion and String-Number Operations
The Type Error Problem

Cannot directly concatenate strings and integers using + operator
"Happy " + 23 causes TypeError: Cannot concatenate string and int
Python treats + differently based on data types:
Between two strings: concatenation
Between two numbers: addition
Converting Numbers to Strings

Use str() function to convert integers to strings before concatenation
Example: "Happy " + str(23) + " birthday" →

"Happy 23 birthday"
This allows mixing text and numbers in output
Lists: Storing and Managing Multiple Values
What is a List?

Lists store multiple values in a single variable using square brackets []
Example: bicycles = ["Trek", "Cannondale", "Red Line", "Specialized"]
An empty list: bicycles = []
Accessing List Elements (Indexing)

Python uses zero-based indexing (counting starts at 0)
Positive indexing: 0, 1, 2, 3... from the beginning
Negative indexing: -1, -2, -3... from the end
-1 refers to the last element
-2 refers to the second-to-last element
Examples:

bicycles[0] → "Trek"
bicycles[3] → "Specialized"
bicycles[-1] → "Specialized"
bicycles[-2] → "Red Line"
Modifying Lists: Adding and Changing Elements
Replacing an Element (Assignment)

Direct assignment overwrites an existing value
Syntax: list_name[index] = new_value
Example: motorcycles[0] = "Ducati" replaces "Honda" with "Ducati"
Does NOT add a new element, only replaces the existing one
Appending Elements (Adding to End)

.append() method adds a single element to the end of the list
Syntax: list_name.append(value)
Example: motorcycles.append("Ducati") adds "Ducati" at the end

Useful for building lists dynamically (e.g., collecting user input)
Inserting Elements at Specific Positions

.insert() method adds an element at a specified index without replacing
Syntax: list_name.insert(index, value)
Example: motorcycles.insert(0, "Ducati") adds "Ducati" at the beginning
All subsequent elements shift right by one position
Removing Elements from Lists
Deletion Using del Statement

Removes an element at a specified index permanently
Syntax: del list_name[index]
Example: del motorcycles[0] removes the first element
The element is gone; cannot be recovered or

stored
Removing by Using .pop() Method

Removes and returns an element (allows storage of removed value)
Default: removes the last element
Syntax without index: list_name.pop()
Syntax with index: list_name.pop(index)
Example: first_owned = motorcycles.pop(1) removes element at index 1 and stores it
Can be stored in a variable: removed_item = motorcycles.pop(0)
Removing by Value Using .remove() Method

Removes the first occurrence of a specified value
Syntax: list_name.remove(value)
Example: motorcycles.remove("Ducati") removes the first "Ducati" from the list
Does not remove all occurrences; only the first

instance
To remove all occurrences, a loop is required (topic for future sessions)
Comparison Summary:

Method	Removes	Stores Value	Permanent
del	By index	No	Yes
.pop()	By index or default end	Yes	Yes
.remove()	By value	No	Yes

Organizing and Managing Lists
Sorting Lists

.sort() method: Permanently sorts the list in alphabetical order

Syntax: list_name.sort()
Add reverse=True for reverse alphabetical order: list_name.sort(reverse=True)
Affects the list permanently for the rest of the program

sorted() function: Temporarily displays list in sorted order without changing original

Syntax: sorted(list_name)
Add reverse=True for reverse order:
sorted(list_name, reverse=True)
Does not modify the actual list
Reversing List Order

.reverse() method: Reverses the current order of elements
Syntax: list_name.reverse()
Useful for viewing latest additions (like newest users in a dynamic list)
Calling twice returns list to original order
Finding List Length

len() function: Returns the number of elements in a list
Syntax: len(list_name)

Example: len(bicycles) returns 4
Useful for dynamic lists to track the number of items (e.g., registered users)
Index Errors and List Boundaries
Understanding Index Out of Range Error

Occurs when attempting to access an index that doesn't exist
Error message: IndexError: list index out of range
Example: List has 3 elements (indices 0, 1, 2), but accessing index 3 causes error
Accessing an empty list at any index causes this error
Prevention Tips

Remember: Valid indices go from 0 to len(list) - 1
For negative indexing: -1 to -len(list)
Always verify the list length before accessing elements
Code Comments and Best Practices

Using Comments

Comment syntax: # comment text
Everything after # on a line is ignored by Python
Comments explain the purpose and logic of code
Importance of Comments

Essential for understanding complex code (10,000+ lines)
Helps when others need to read or modify the code
Professional standards (Google, Microsoft, Siemens) have specific comment guidelines
Effective comments should explain what the code does and how it works
Should not simply restate obvious code
Practical Exercise: Shipment Inventory Management
Task: Write Python code for the following sequence of operations on a hardware inventory

list:

Create a list: starting_inventory = ["router", "switch", "firewall", "server", "cable"]
Remove "server" using .remove() method
Insert "fiber_optic" at the beginning using .insert()
Replace "cable" (last element) with "USB_hub" using index assignment
Pop the second item and store it in allocated_device variable
Append "rack_mount" to the end using .append()
Delete the item at index 2 using del statement
Print the final inventory and allocated_device value
Key Skills Assessed:

List creation and initialization
Correct use of indexing and method syntax
Understanding the difference between various

removal methods
Proper element insertion and replacement
Output verification
Common Pitfalls and Important Notes
String vs. Number Concatenation

Python distinguishes between + for string concatenation and + for arithmetic addition
Error type depends on which operand is encountered first
Always use str() conversion when mixing strings and numbers
Mutable Nature of Lists

Lists can be modified after creation
Changes made to lists persist unless explicitly reset
Methods like .sort() permanently alter the list
Zero-Based Indexing

Python counts from 0, not 1
This is a common source of "off-by-one" errors for beginners
Negative indexing provides convenient access to end elements
Next Topics to Explore
Advanced list operations with loops
Conditional statements (if statements)
Further list comprehension techniques
Working with multiple data types in collections

Lecture 20

Using If Statements with Lists
This lecture covers the third major topic of the chapter on if statements, focusing on how to effectively use conditional statements when working with lists in Python.

Overview

The session discusses three practical applications of if statements with lists:

Checking for special items within a list
Verifying whether a list contains data before processing
Working with multiple lists simultaneously to validate data
Checking for Special Items
Concept: Use if statements inside for loops to handle items that require special treatment differently from others
Example: In a pizzeria application, when a customer requests toppings, the code can check if a specific ingredient is available before confirming it will be added
Implementation:
Create a for loop that iterates through requested toppings
Add an if statement inside the loop to check for

unavailable items
Print a special message if an ingredient is out of stock (e.g., "Sorry, we are out of green peppers right now")
Print the default confirmation message for available items
Checking That a List is Not Empty
Importance: When accepting user input, lists may be empty, so validation is necessary before processing
Key Concept: When a list name is used in an if statement, Python returns True if the list contains at least one item, and False if the list is empty
Implementation:
Write an if statement using just the list name: if requested_toppings:
Place the for loop inside the if block to process only non-empty lists
Use an else clause to handle empty lists with an

appropriate message
Example: Check if a pizza topping list is empty before building a pizza. If empty, prompt the user to confirm they want a plain pizza.
Using Multiple Lists
Purpose: Validate user requests against a list of available options to ensure data integrity
Concept: Compare items from one list against another list to verify availability or uniqueness
Implementation:
Create two lists: one for available items and one for requested items
Use nested loops and if statements to check each requested item against the available items list
Use the in operator to check if a requested item exists in the available items list
Example: Before adding a topping to a pizza, check if it exists in the available toppings list. If the topping (like french fries) is not available,

print a rejection message.

Key Takeaways from If Statements Chapter

Conditional tests always evaluate to either true or false

Simple if statements, if-else statements, and if-elif-else chains provide different ways to handle conditions

These structures allow identification of specific conditions and determine when they have been met in programs

If statements combined with loops enable efficient handling of certain items in lists while maintaining code efficiency

Lists automatically evaluate to true or false based on whether they contain items, enabling simple validation checks

Next Topic

The upcoming session will introduce Python dictionaries, which are similar to lists but allow connection of related pieces of information.

Dictionaries can be looped through and combined with lists and if statements to model more complex real-world situations.

Lecture 21 Introduction to Dictionaries in Python
This lecture introduces dictionaries as a fundamental data structure in Python for storing and managing collections of key-value pairs, enabling more accurate modeling of real-world objects and information.

Overview of Dictionaries
A dictionary in Python is a collection of key-value pairs enclosed in curly braces {}
Each key is connected to a value, and keys are used to access their associated values
Dictionary values can be strings, numbers, lists, or even other dictionaries
Dictionaries are dynamic structures that can be modified at any time

Working with Dictionaries

Dictionary Syntax and Structure

Dictionaries use curly braces {} instead of square brackets used for lists

Key-value pairs are separated by colons :, with each key-value pair separated by commas

Keys must be unique within a dictionary

Example: alien_0 = {'color': 'green', 'points': 5}

Accessing Dictionary Values

Access values using the key name in square brackets: dictionary_name[key]

Example: alien_0['color'] returns 'green'

When concatenating dictionary values with strings in print statements, use str() to convert non-string values to strings

Adding New Key-Value Pairs

Add new pairs by specifying the dictionary name, new key in square brackets, and assigning a value

Example: alien_0['x_position'] = 0

Multiple key-value pairs can be added sequentially
Dictionary output will include all pairs, both original and newly added
Starting with an Empty Dictionary
Create an empty dictionary using empty curly braces: dictionary_name = {}
Add key-value pairs one at a time to the empty dictionary
Useful when storing user-supplied data or generating large numbers of key-value pairs automatically
Modifying Values in a Dictionary
Modify existing values by reassigning the value to an existing key
Example: alien_0['color'] = 'yellow' changes the color from green to yellow
The new value replaces the old value permanently
Conditional statements (if/elif/else) can

determine which values to assign based on specific conditions

Removing Key-Value Pairs

Use the del statement to permanently remove a key-value pair

Syntax: del dictionary_name[key]

Once deleted, a key-value pair cannot be recovered in subsequent code

Example: del alien_0['points'] removes the points key and its associated value

Dictionary of Similar Objects

Store information about multiple objects using a single dictionary

Example: favorite_languages = {'jen': 'python', 'sarah': 'c', 'edward': 'ruby', 'phil': 'python'}

Dictionaries can be formatted across multiple lines for better readability

Break dictionary definitions after the opening brace and place each key-value pair on a separate line

Print statements can also be broken across multiple lines as long as the opening and closing parentheses are matched

Formatting Dictionary Output

Use string concatenation with the + operator to combine strings with dictionary values

Use the .title() method to capitalize the first letter of words

Insert blank lines between outputs using new line characters for cleaner formatting

Dictionary values and print statements can span multiple lines for improved code readability

Practice Exercises

Three exercises are provided to reinforce dictionary concepts:

Personal Information Dictionary - Create a dictionary storing personal details (first name, last name, age, city) and print each piece of information

Favorite Numbers Poll - Store five people's favorite numbers in a dictionary and print each person's name with their favorite number

Programming Glossary - Create a dictionary with five programming terms as keys and their meanings as values; format output neatly with proper spacing and indentation between word-meaning pairs

Lecture 22
Looping Through a Dictionary in Python
This session focused on the second topic of the Dictionaries chapter: looping through dictionaries. The instructor covered four main subtopics with practical examples demonstrating how to traverse and access dictionary data efficiently using different methods.

Overview of Dictionaries and Looping

A Python dictionary can contain a few key-value pairs or millions of pairs. Since dictionaries often hold large amounts of data, Python provides multiple methods to loop through them. The main approaches include looping through key-value pairs, keys only, keys in order, or values only.

Looping Through All Key-Value Pairs

Method: Use the .items() function with a for loop

Syntax: for key, value in dictionary_name.items():

How it works: The .items() method stores both the key and its associated value into separate variables during each iteration

Example: In a favorite_languages dictionary, a for loop can access each person's name (key) and their programming language preference (value) simultaneously

Variable naming: You can use any variable names instead of "key" and "value" (e.g., "name" and "language") to make the code more readable
Looping Through All Keys in a Dictionary
Primary method: Use the .keys() function
Syntax: for name in dictionary_name.keys():
Important note: Using .keys() is actually optional because Python treats looping through a dictionary as looping through its keys by default
Simplified syntax: for name in dictionary_name: achieves the same result without explicitly calling .keys()
Practical use: Useful when you only need to work with keys and don't require the associated values
Checking key existence: The .keys() method can be combined with conditional statements to check if a specific key exists in the dictionary
Looping Through Dictionary Keys in Order
Method: Use the sorted() function combined with .keys()

Syntax: for name in sorted(dictionary_name.keys()):
Functionality: The sorted() function creates a copy of the keys and arranges them in alphabetical order before looping
Benefit: Allows you to display dictionary contents in a specific order without modifying the original dictionary
Process: Python lists all keys from the dictionary, sorts them alphabetically, and then loops through the sorted list
Looping Through All Values in a Dictionary
Method: Use the .values() function
Syntax: for language in dictionary_name.values():
Purpose: Access only the values without needing to reference the keys
Limitation: This approach includes duplicate values if multiple keys share the same value
Handling Duplicate Values with Sets
Problem: When many entries exist, using

.values() results in repetitive output
Solution: Use the set() function to filter duplicate values
Syntax: for language in set(dictionary_name.values()):
How sets work: A set is similar to a list but only contains unique items; each value appears only once
Advantage: Provides a cleaner output showing only distinct values from the dictionary
Practical Applications
The instructor demonstrated real-world scenarios such as:

Creating conditional statements within loops to perform actions based on specific keys
Combining dictionaries and lists to filter and process data
Displaying personalized messages based on dictionary contents

Checking whether specific entries exist in a dictionary before processing them

Exercises for Practice

Students were assigned three exercises to reinforce learning:

Glossary exercise: Create a dictionary with five programming terms as keys and their definitions as values, then loop through to display them. Expand the glossary to ten terms.

Rivers exercise: Build a dictionary of three major rivers and the countries they flow through, then use loops to print sentences about each river and separately list all rivers and countries.

Poll survey exercise: Create a list of people who should take a poll, including some already in the dictionary and some new names. Use conditional loops to thank those who responded

and invite those who haven't yet.

Lecture 23
List of Dictionaries
Storing Multiple Dictionaries in a List

A list of dictionaries is useful when you need to manage multiple similar objects, such as a fleet of aliens in a video game
Each alien is represented as a dictionary containing properties like color, points, and speed
Multiple alien dictionaries are stored together in a single list named "aliens"
Creating Dictionaries Dynamically

Use a for loop with the range() method to generate multiple dictionaries automatically
The append() method adds each new dictionary to the list

Example: Creating 30 green aliens with range(30) iterates the loop 30 times, creating one new alien dictionary per iteration
Modifying Nested Dictionaries

Use for loops combined with if statements to modify specific dictionary values based on conditions
You can change properties like color, points, and speed for aliens that meet certain criteria
Use list slicing (e.g., [0:5]) to view a subset of dictionaries in the list
List in a Dictionary
Storing Multiple Values for a Single Key

A dictionary value can be a list when you need to associate multiple items with one key
Example: A pizza dictionary has a "crust" key with a single string value and a "toppings" key with a list value containing multiple topping

items
This structure is useful when people can have multiple preferences, such as multiple favorite programming languages
Accessing List Items Within Dictionaries

Access the entire list using the dictionary name and key: pizza['toppings'] returns the full list
Use nested for loops to iterate through individual items in the list
The outer loop iterates through dictionary items, and the inner loop iterates through list items
Nested Loops with Dictionary Items

Use the .items() method in the outer loop to get key-value pairs
In the inner loop, iterate through the list values associated with each key
This allows printing formatted output for each person and their multiple favorite languages

Dictionary in a Dictionary
Nesting Dictionaries as Values

A dictionary can have another dictionary as its value
Example: A "users" dictionary contains usernames as keys, with each username having an associated dictionary containing user information like first name, last name, and location
This structure is ideal for storing complex, multi-level information about multiple entities
Accessing Nested Dictionary Values

Use the .items() method to iterate through the outer dictionary
Store the inner dictionary in a variable, which then becomes accessible as a dictionary itself
Access values from the nested dictionary using square bracket notation with the appropriate key

Working with Multiple Levels

The outer loop iterates through usernames and their associated dictionaries
Within each iteration, access specific values from the nested dictionary using keys like "first," "last," and "location"
This allows generating formatted output that combines information from multiple levels
Key Takeaways
Nesting enables organization: Multiple related objects can be stored and accessed efficiently
List of dictionaries: Best for managing collections of similar items with multiple properties
List in a dictionary: Ideal when a single key needs multiple associated values
Dictionary in a dictionary: Useful for storing complex, hierarchical information
Nested loops: Required to access and process

deeply nested data structures.

Lecture 24

Python Lists: Operations, Control Flow, and Data Manipulation

Overview of Lecture Content

This lecture continued the discussion on Python programming, focusing on advanced list operations using loops, conditional statements, and practical applications. The session emphasized understanding code logic and interpretation rather than just syntax, with a comprehensive problem-solving example involving nested lists and multiple filtering operations.

Looping Through Lists

Understanding the Need for Loops

Loops are essential for handling repetitive tasks efficiently. Instead of writing individual code for

each element (e.g., checking 250 numbers for prime or composite properties), loops automate the process.

For Loop Syntax:

The for loop iterates through each element in a list
Python uses indentation (not curly braces) to define loop scope
A colon (:) marks the beginning of the loop block
Indentation is critical; improper spacing results in an "IndentationError"
List Iteration Example

```
magicians = ['alice', 'david', 'carolina']
for magician in magicians:
print(magician.title())
```

Each iteration assigns the next list element to the loop variable, allowing sequential processing.

Multiple Statements in Loops

Multiple statements can be included within a loop body, all executed for each iteration as long as they maintain proper indentation.

Creating Numerical Lists

Using the range() Function

The range() function generates sequences of numbers:

range(1, 6) produces: 1, 2, 3, 4, 5 (excludes 6)
range(2, 11, 2) produces: 2, 4, 6, 8, 10 (generates even numbers with step of 2)

Converting to Lists

The list() function converts range objects into list structures:

numbers = list(range(1, 6)) # Results in [1, 2, 3, 4, 5]

Statistical Operations on Lists

min(list) - Returns the minimum value
max(list) - Returns the maximum value
sum(list) - Calculates the total
List Comprehensions
List comprehensions provide concise syntax for creating and modifying lists in a single line:

squares = [value**2 for value in range(1, 11)]
This replaces traditional loop structures with more readable, efficient code.

Practical Application: Student Records Management
Problem Overview
A comprehensive example involved managing nested lists (lists containing lists) with multiple operations:

Data Structure:

```
student_records = [
['Alice', [75, 88, 92]],
['Bob', [65, 70, 72]],
['Charlie', [88, 90, 95]],
['David', [78, 82, 85]],
['Eve', [45, 50, 55]]
]
```

Operation 1: Removing Underperformers

Students with average scores below 60 were filtered out:

Calculate average using: sum(scores) / len(scores)

Create new list for eligible students

Use conditional statements to filter results

Key Concept: Avoid modifying lists while iterating. Instead, create a new list and reassign.

Operation 2: Rewarding Top Scorers

Students with any subject score of 90 or above

received a "distinction" designation:

Used max(scores) to find highest individual score
Added distinction marker to qualified students
Operation 3: Prioritized Enrollment
New student data was inserted at the beginning of the list:

Used insert(0, new_student) instead of append() for first position placement
append() adds to end; insert() allows position specification
Operation 4: Sorting and Display
Results were sorted alphabetically by name:

student_records.sort(key=lambda x: x[0])
The lambda function specified sorting criteria (first element/name).

List Slicing

Slice Syntax and Examples

list[start:end] - Retrieves elements from start index to end-1

list[:3] - Starts from beginning (index 0) to index 2

list[1:] - Starts from index 1 to end of list

list[1:4] - Retrieves elements at indices 1, 2, and 3

Application: Useful for dividing large datasets into training and testing subsets in machine learning.

Looping Through Slices

Loops can iterate through specific list portions:

```
for player in players[:4]:
print(player)
```

Copying Lists

Method 1: Shallow Copy (Reference Assignment)

```
new_list = original_list
```

Both variables point to the same memory

location; changes affect both.

Method 2: Deep Copy

new_list = original_list[:]

Creates independent list with separate memory allocation; changes are isolated.

Tuples: Immutable Sequences

Tuple Characteristics

Tuples use parentheses and are similar to lists but cannot be modified after creation:

```
dimensions = (200, 50)
print(dimensions[0]) # Output: 200
dimensions[0] = 250 # Error: tuple object does not support item assignment
```

Tuple Operations

Accessing elements uses same indexing as lists

Looping through tuples uses identical syntax

Modification requires complete re-initialization

Conditional Statements: If-Elif-Else

Boolean Values and Comparison Operators

Conditional statements return Boolean values (True/False):

== (equality) - Comparison operator

= (assignment) - Assignment operator

!= (not equal) - True if values differ

<, >, <=, >= - Relational operators

Case Sensitivity:

String comparisons are case-sensitive

Use .lower() or .upper() methods for case-insensitive comparisons

Logical Operators

and - Both conditions must be True

or - At least one condition must be True

not - Reverses Boolean value

in - Checks if value exists in list

If-Elif-Else Chain

```
if age <= 4:
price = 0
elif age <= 18:
price = 5
else:
price = 10
```

Key Difference: If-elif-else stops after first True condition, while multiple if statements check all conditions regardless.

Assessment and Course Expectations

Evaluation Focus

Final assessments emphasize:

Code interpretation skills - Understanding logic flow

Code readability - Ability to explain code in plain English

Error identification - Recognizing indentation and logical errors

Pen-and-paper coding with flexible evaluation (focus on logic, not strict syntax)

Planned Assessments

One quiz scheduled for end of October

No third quiz planned

Assignments will require both code and detailed explanations

Descriptive questions primarily from foundational topics; coding sections focus on interpretation

Key Takeaways

Indentation is fundamental to Python syntax and readability

Nested lists enable complex data structures

List comprehensions provide efficient, concise syntax

Slicing is essential for data subset operations

Conditional logic directs program flow based on conditions

Tuples offer immutability when constant data is

required
Understanding code logic is prioritized over syntax memorization

Lecture 25

Foundations of Computing: User Input and While Loops

Overview

This lecture introduces methods for accepting user input and implementing while loops in Python. The session covers how programs interact with users by collecting information, processing numerical data, and controlling program flow using loop constructs. Two main types of loops are compared: the familiar for loop and the new while loop.

Topic 1: How the Input Function Works

Understanding the Input Function

The input() function pauses program execution and waits for user input
It accepts one string argument, which serves as a prompt displayed to the user
The user's response is stored as a string in a variable
Example: message = input("Tell me something and I will repeat it back to you")
Writing Clear Prompts
Prompts should be clear and easy to understand so users know what to enter
Prompts guide the user on what information is needed
Multi-line prompts can be created by storing a prompt in a variable and concatenating strings
Example:
prompt = "If you tell us who you are, we can personalize the message you see.
"

prompt += "What is your first name? "

name = input(prompt)

Using the int() Method for Numerical Input

The input() function treats all user input as strings, even if numbers are entered

To perform mathematical operations or comparisons, string input must be converted to integers using the int() method

Without conversion, Python cannot compare strings to numbers and will produce an error

Example: age = int(input("How old are you? "))

The int() function converts string representations of numbers to integer values

Once converted, numerical operations and comparisons can be performed

If non-integer values (like decimals) are entered, a ValueError will occur

The Modulo Operator (%)

The modulo operator divides one number by another and returns the remainder

Syntax: number % divisor

Useful for determining if a number is even or odd
Example: if number % 2 == 0: checks if a number is even (remainder is 0)
Example: if number % 2 != 0: checks if a number is odd (remainder is not 0)
Differs from regular division (/) which returns the quotient
Topic 2: Introducing While Loops
Understanding While Loops
A while loop runs continuously as long as a specified condition is true
Unlike for loops that iterate a predetermined number of times, while loops depend on condition evaluation
Structure requires three components:
Initialize: Set the starting value of a variable
Condition: Specify when the loop should continue
Update: Increment or modify the variable each iteration
While Loop Structure and Execution

Syntax:

```
current_number = 1
while current_number <= 5:
print(current_number)
current_number += 1
```

The += operator is shorthand (e.g., current_number += 1 equals current_number = current_number + 1)

The loop checks the condition before each iteration

If the condition is true, the code block executes; if false, the loop terminates

Example output for the above code: 1, 2, 3, 4, 5

Letting the User Control Program Flow

Programs can continue running until a user chooses to exit

A quit value is established to terminate the loop

Example implementation:

```
prompt = "Tell me something: "
prompt += "
```

Enter 'quit' to end the program: "
message = ""

while message != "quit":
message = input(prompt)
if message != "quit":
print(message)

The if statement prevents the quit command itself from being printed as output

The program continues requesting input until the user enters the quit value

Practice Exercises

Write a program asking the user what rental car they prefer and print a personalized message

Create a program that asks for dinner group size and informs users if they must wait (more than 8 people) or if their table is ready

Develop a program that takes a number as input and determines whether it is a multiple of 10

Upcoming Topics

The following topics will be covered in the next session:

Using flags to control loops
Using break to exit loops
Using continue to skip iterations
Avoiding infinite loops

Lecture 26

Foundations of Computing: User Input and While Loops

Overview

This lecture covers the continuation of the chapter on user input and while loops, focusing on advanced loop control techniques and practical applications with data structures. The session builds upon previous discussions of input functions and basic while loop mechanics.

Using a Flag

What is a Flag?

A flag is a variable (typically a Boolean) that controls program flow and determines whether the program should continue running
Flags help manage complex programs where multiple conditions could terminate execution
Common flag values are True (program active) or False (program stops)
Why Use Flags?

In complex programs like games, many events can trigger program termination (e.g., player runs out of ships, time runs out, protected cities destroyed)
Instead of testing multiple conditions in a single while statement, a flag monitors overall program status
Makes code cleaner and more maintainable
Practical Implementation

Set flag variable (e.g., active = True) before the while loop
Write condition as while active:
Change flag to False when an exit condition is met
When flag becomes False, the while loop terminates automatically
Using Break to Exit a Loop
Break Statement Fundamentals

The break statement immediately exits a loop without executing remaining code
Provides direct flow control for exiting loops based on conditional tests
How Break Works

When Python encounters break, it exits the current loop entirely
Often used with while True: to create loops that

run until a specific condition is met
The loop terminates immediately upon encountering break, regardless of the loop condition
Example Application

Prompt user to enter city names until they type "quit"
When "quit" is entered, the if statement triggers break
Python exits the loop and continues with code after the while block
Using Continue in a Loop
Continue Statement Purpose

The continue statement skips remaining code in the current iteration and returns to the loop's beginning
Useful for filtering or processing specific items within a loop

How Continue Works

When encountered, continue immediately goes back to the loop condition check
The rest of the loop body is skipped for that iteration
The loop continues normally with the next iteration
Practical Example

Loop counts from 1 to 10 but prints only odd numbers
For even numbers (divisible by 2), continue skips the print statement
For odd numbers, the print statement executes normally
Avoiding Infinite Loops
What Creates Infinite Loops

A loop with no exit condition (e.g., while True:

without break)
Forgetting to increment or modify a counter variable
Missing flag updates that would terminate the loop
How to Prevent Infinite Loops

Every while loop must have a termination mechanism:
A counter that eventually makes the condition false
A flag that changes to False
A break statement that executes
Double-check loop logic before running code
Handling Accidental Infinite Loops

In Google Colab, click the pause button to stop execution
Close the terminal window if running in command line

Review loop conditions to identify and fix the issue
Using While Loops with Lists and Dictionaries
Moving Items from One List to Another
Use Case

Transfer unverified users to a verified users list after confirmation
Move items between lists based on processing or validation
Implementation Steps

Create a list of unconfirmed items and an empty list for confirmed items
Use while loop with the condition while unconfirmed_users:
Use .pop() to remove items from the unconfirmed list
Append each item to the confirmed list using .append()

Loop automatically terminates when the source list is empty
Advantage Over For Loops

Unlike for loops, while loops safely modify lists during iteration
Python doesn't lose track of items when using while with .pop()
Removing All Instances of a Value from a List
Problem Statement

Remove every occurrence of a specific value from a list
Useful when an item appears multiple times at different positions
Solution Using While Loop

Use condition while value in list:
Call .remove(value) within the loop
Loop continues until all instances are removed

When .remove() finds and deletes the value, the loop checks again
Terminates when the value no longer exists in the list
Example Application

Pets list contains: dog, cat, dog, goldfish, cat, rabbit, cat
Use while "cat" in pets: followed by pets.remove("cat")
Result: only dog, dog, goldfish, rabbit remain
Filling a Dictionary with User Input
Purpose

Collect user input and store related information as key-value pairs
Use dictionaries to connect responses with participant information
Implementation Process

Create an empty dictionary (e.g., responses = {})
Set a flag to control the polling loop (e.g., polling_active = True)
Use while polling_active: to prompt for data
Collect user input for both key (name) and value (response)
Store data as dictionary[key] = value
Ask if user wants to continue; update flag based on response
Use a for loop to display results: for key, value in dictionary.items():
Example Application

Prompt: "What is your name?" and "Which mountain would you like to climb?"
Store as: responses[name] = mountain
Display results showing each person's mountain preference
Key Takeaways
User input allows programs to accept text and

numerical data from users
While loops enable programs to run as long as users require
Flow control with flags, break, and continue provides flexible loop management
While loops with data structures safely modify lists and populate dictionaries during iteration
Multiple techniques (flags, break, continue) offer different solutions for controlling loop execution
Proper loop termination prevents infinite loops and program freezes
Next Chapter Preview
The upcoming chapter will cover Functions, which allow breaking programs into manageable, reusable parts with specific purposes. Functions improve code efficiency, maintainability, and reusability across multiple programs.

Lecture 27

Introduction to Functions in Python

This lecture introduces the concept of functions in Python as reusable blocks of code designed to perform specific tasks. Functions improve code organization, readability, and maintainability by eliminating the need to rewrite logic multiple times throughout a program.

Overview of Functions

Function Definition: A function is a named block of code that performs one specific job

Code Reusability: Functions allow you to write code once and call it multiple times, reducing errors and improving debugging

Benefits: Programs become easier to write, read, test, and maintain

Key Concept: Instead of repeating code, you simply call the function by name wherever needed

Defining a Function

Syntax: Use the DEF keyword followed by the function name, parentheses, and a colon
Example: def greet_user():
Function Body: All indented lines following the function definition constitute the body where logic code is written
Docstring: A comment between triple quotes that describes what the function does
Example: """Display a simple greeting."""
Function Call: Execute the function by writing its name followed by parentheses
Example: greet_user()
Parameters and Arguments
Parameters: Variables listed in the function definition (e.g., username in def greet_user(username):)
Arguments: Actual values passed to the function when calling it (e.g., "Arun" in greet_user("Arun"))
Terminology: Parameters and arguments are used interchangeably, though technically

parameters are in the definition and arguments are in the function call
Requirement: The number of arguments passed must match the number of parameters defined (unless default values are used)
Passing Arguments: Positional Arguments
Definition: Arguments must be provided in the same order as parameters are defined in the function
Order Matters: The sequence of arguments is critical; mismatching order produces incorrect results
Example: describe_pet("hamster", "Harry") assigns "hamster" to animal_type and "Harry" to pet_name
Multiple Parameters: Functions can have multiple parameters, and all arguments must be supplied in the correct order
Error Handling: Providing fewer arguments than required parameters results in a TypeError

Passing Arguments: Keyword Arguments
Definition: Name-value pairs passed to a function where each argument explicitly associates a parameter name with its value
Example: describe_pet(animal_type="parrot", pet_name="Kyle")
Flexibility: Order does not matter when using keyword arguments because Python matches names to values
describe_pet(pet_name="Kyle", animal_type="parrot") produces the same result as above
Clarity: Keyword arguments make code more readable by clarifying the role of each value
Usage: Keyword arguments free programmers from worrying about argument order
Default Values
Purpose: Assign a default value to a parameter so the argument becomes optional during function calls

Example: def describe_pet(pet_name, animal_type="dog"):
Usage: If no argument is provided for a parameter with a default value, Python uses the default
Overriding: An explicit argument in the function call overrides the default value
Parameter Order Rule: Parameters with default values must be listed after parameters without default values
Incorrect: def func(param1="default", param2):
Correct: def func(param1, param2="default"):
Simplified Calls: Default values reduce the number of arguments required in function calls
Equivalent Function Calls
Multiple ways to call the same function can produce identical results:

Positional Arguments Only: describe_pet("Willie", "dog")

Keyword Arguments Only: describe_pet(pet_name="Willie", animal_type="dog")
Mixed Arguments: describe_pet("Willie", animal_type="dog")
Reordered Keywords: describe_pet(animal_type="dog", pet_name="Willie")
Using Defaults: describe_pet("Willie") uses the default value for animal_type
Avoiding Argument Errors
Mismatched Arguments: Errors occur when the number of arguments provided does not match the function's expectations
Too Few Arguments: Providing fewer arguments than required parameters raises a TypeError indicating missing required positional arguments
Too Many Arguments: Providing more arguments than the function accepts raises a TypeError indicating too many positional

arguments
Traceback Window: Python displays helpful error messages showing the location of the problem and which arguments are needed
Descriptive Naming: Using clear, logical names for functions and parameters makes error messages more useful and code easier to understand
Practice Exercises
Students are assigned five exercises to practice function concepts:

Create a display_message() function that prints a sentence about chapter content
Write a favorite_book() function with a title parameter and call it with a book title argument
Create a make_shirt() function with size and message parameters; call it using positional arguments once and keyword arguments once
Modify make_shirt() to have default values

(large size and "I love Python" message); call it for different size and message combinations
Write a describe_city() function with city name and state parameters (state with a default value); call it for three different cities with at least one non-default state
Upcoming Topics
The next session will cover:

Return values from functions
Passing lists to functions
Passing an arbitrary number of arguments

Lecture 28

Second Lecture on Functions: Return Values, Optional Arguments, and List Management

Overview
This lecture continues the exploration of functions in Python, focusing on return values,

optional arguments, dictionaries, loops within functions, and list manipulation. The session demonstrates how functions can process data and return values to the calling code, making programs more efficient and maintainable.

Return Values
What are Return Values?

A return value is data that a function sends back to the line of code that called the function
Functions can process data internally and return results without printing directly
The return statement transfers a value from inside the function to the function call
Example: Formatting Names

Function get_formatted_names() takes two parameters: first name and last name
Combines names with a space and converts to

title case
Returns the formatted full name to be stored in a variable
The returned value can be printed or stored for later use
Benefits:

Simplifies the main body of the program
Allows results to be stored and used elsewhere in the code
Making Arguments Optional
Purpose:

Some function arguments should be optional to handle varying input scenarios
Useful when users may not always provide complete information (e.g., not everyone has a middle name)
Implementation:

Assign a default value to a parameter (e.g., empty string for middle name)
Optional parameters must be placed at the end of the parameter list
Use conditional logic (if/else) to handle cases where optional arguments are provided or omitted
Example: Three-Part Names

Function accepts first name, last name, and optional middle name
If middle name is provided, include it in the full name
If not provided, return only first and last names
Function handles both scenarios without errors
Advantage:

Increases function flexibility and usability
Reduces errors when incomplete information is provided

Returning a Dictionary
Concept:

Functions can return complex data structures like dictionaries
Useful for returning multiple pieces of related information at once
Example: Building a Person Dictionary

Function build_person() takes first and last name parameters
Creates a dictionary with keys "first" and "last"
Returns the entire dictionary to the function call
Returned dictionary is stored in a variable for use
Extending the Function:

Can add optional parameters like middle name, age, or occupation
Use conditional statements to include optional information in the returned dictionary

Default values (e.g., empty string) make additional parameters optional
Use Case:

Efficiently organize and return multiple related data points
Simplifies passing structured information between different parts of the program
Using Functions with While Loops
Integration:

Functions work seamlessly with Python control structures including while loops
Combines loop functionality with function reusability
Example: Interactive Greeting Program

Function get_formatted_name() formats user input
While loop repeatedly requests user input

Break statement exits the loop when user enters "Q"
Uses conditional statements to check for exit condition
Implementation Details:

Use break keyword to terminate loops based on specific conditions
Comparison operator (==) checks if input equals "Q"
Assignment operator (=) assigns values to variables
Program continues until user chooses to quit
Passing a List to a Function
Concept:

Lists can be passed as arguments to functions
Function gains direct access to list contents
Useful for processing multiple items of the same type

Example: Greeting Multiple Users

Function greet_users() accepts a list of names
For loop iterates through each name in the list
Personalizes a greeting message for each user
Prints formatted greeting for all users
Advantages:

Handles variable numbers of items
Eliminates need to write separate code for each item
Keeps code organized and reusable
Modifying a List in a Function
Key Principle:

Changes made to a list inside a function are permanent
The function modifies the original list, not a copy
Allows efficient processing of large amounts of data

Example: 3D Printer Design Queue

Maintains two lists: unprinted designs and completed models
Uses pop() function to remove items from the end of unprinted list
Uses append() function to add items to completed models list
Original list is permanently modified
Organizing with Functions:

print_models() function handles the printing and transfer logic
show_completed_models() function displays results
Separates concerns: each function has one specific job
Improves code readability and maintainability
Benefits of Multiple Functions:

Easier to extend and maintain code
Can call functions multiple times without rewriting logic
Modifications needed only in one location
Each function focuses on a single task
Preventing a Function from Modifying a List
Problem:

When modifying a list in a function, the original list becomes empty after transfer
May need to preserve the original list for records or future use
Solution: Passing a Copy

Use slice notation list_name[:] to pass a copy instead of the original
Colon between square brackets indicates "all elements from first to last"
Function modifies only the copy, leaving the original intact

Implementation:

Original list parameter: unprinted_designs (passed as is)
Copy list parameter: unprinted_designs[:] (passed as copy)
Second list receiving appended values: completed_models (passed as is)
Benefit:

Original data remains safe and unchanged
Preserves information for records or multiple operations
Provides flexibility in how data is processed and stored
Exercises
City-Country Function: Create city_country() function returning formatted strings like "Chennai, India"

Music Album Dictionary:

Create make_album() function building dictionaries with artist and title
Add optional parameter for number of tracks
Generate at least three album dictionaries
Interactive Album Entry:

Use while loop for user input
Collect artist name and album title
Call make_album() with user input
Include quit option
Magician List:

Create list of magician names
Pass to show_magicians() function that prints each name
Modify Magician Names:

Create make_great() function adding "the Great"

to each name

Modify original list

Display modified list

Preserve Original List:

Call make_great() with copy of magician list

Store modified list separately

Display both original and modified lists

Upcoming Topics

Passing an arbitrary number of arguments

Storing functions in modules